

```

00 EB 3C 90 JMP EB 3C 90
03 4D 53 34 4F05 OEM NAME
0B 0201 BYTES PER SECTOR
0D 01 SECTORS PER CLUSTER
0E 02 RESERVED SECTORS
10 02 NUMBER OF FATs
11 00E0 ROOT ENTRIES
13 2880 TOTAL SECTORS (16)
15 F0 MEDIA DESCRIPTOR
16 009 SECTORS PER FAT
18 12 SECTORS PER TRAILER
1A 02 SECTORS PER HEAD
1C 0000 HIDDEN SECTOR
20 0000000 TOTAL SECTORS
24 80 NUMBER OF HEADS
25 00 SECTORS PER TRACK
26 29 BOOT SIGNATURE
27 12345678 VOLUME SERIAL NUMBER
2B 4E414M4E020 VOLUME LABEL
36 444AT313220 FILE SYSTEM TYPE
3E 55 AA

```

Understanding and Implementing the Classic File System in C_



Björn Götz

FAT12

Understanding and Implementing the Classic File System in C

Björn Götz

This book is available at <https://leanpub.com/fat12>

This version was published on 2026-10-04



This is a [Leanpub](#) book. Leanpub empowers authors and publishers with the Lean Publishing process. [Lean Publishing](#) is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

© 2026 Björn Götz

*To my beautiful wife, for brightening my life with her endless love; to my mother,
for always believing in me; and to my father, for always being there whenever I
needed him.*

Contents

Introduction	1
Why FAT12?	1
Why Not Just Generate It?	2
Who This Book Is For	2
What We Will Build	3
How This Book Is Structured	3
How to Read This Book	4
Not Production Code	5
About the Author	5
Feedback	6
Prerequisites	7
Operating System and CPU Architecture	7
Command-Line Tools	8
Kernel Tools (Chapter 12 Only)	8
Getting the Code (Optional)	9
Part I: Reading	10
Chapter 1: Hello, Disk!	11
Creating a Disk Image	11
Sectors and LBA	12
The Block Device	13
Debug Logging	15
The File Block Device	16
Putting the Block Device to Work	20
Tracing the Translation	22
What's Next	23
Chapter 2: Mounting the Filesystem	24
Meet the Boot Sector	24

CONTENTS

Reading the Boot Sector	28
Putting It All Together	29
Mounting in Action	31
What's Next	33
Chapter 3: Listing the Root Directory	34
Locating the Root Directory	34
Peeking Inside the Root Directory	38
Anatomy of a Directory Entry	40
The 8.3 Name Format	42
The Attribute Byte	44
The Timestamp Fields	46
Reading the Root Directory	48
Filtering the Entries	48
Putting It All Together	50
The <code>ls</code> Command	53
What's Next	56
Chapter 4: Reading Files	57
The Data Region	57
Carving Up the Data Region	60
Finding a File's First Cluster	61
Resolving a Path	67
Putting It All Together	70
The <code>cat</code> Command	73
What's Next	75
Chapter 5: Reading Multi-Cluster Files	76
When One Cluster Isn't Enough	76
Chaining Clusters with the File Allocation Table	77
Decoding the 12-Bit Entries	81
Loading the FAT	88
Counting the Clusters	90
Reading a Cluster Chain	94
Putting It All Together	96
Finding the Missing B	98
What's Next	99
Chapter 6: Full Path Walking	100
Subdirectories: Files in Disguise	100

CONTENTS

The Path Walking Algorithm	100
Retrofitting Reads	100
What's Next	101
Part II: Writing	102
Chapter 7: Writing Files	103
Finding a Free Slot	103
Encoding the Name	103
Encoding the Timestamp	103
Linking the File	104
Opening a File for Writing	104
Buffering the Write	105
Cluster Allocation	105
Writing the Data	105
Closing the File	106
The <code>create</code> Command	106
What's Next	107
Chapter 8: Creating Directories	108
Assembling the Directory	108
The <code>mkdir</code> Command	108
What's Next	109
Chapter 9: Deleting	110
When Is a Directory Empty?	110
Freeing a Cluster Chain	110
Putting It All Together	110
The <code>rm</code> Command	111
What's Next	111
Chapter 10: Moving (and Renaming)	112
Rename vs. Move	112
Moving the Entry	112
The <code>mv</code> Command	112
What's Next	113
Part III: Advanced	114
Chapter 11: Formatting	115
The Blueprint	115

CONTENTS

The FAT Size Problem	115
The Boot Sector	115
The FAT Region	115
The Root Directory	116
The Data Region	116
Putting It All Together	116
The format Command	117
What's Next	117
Chapter 12: Into the Kernel	118
The RAM Disk Block Device	118
Printing to Screen	118
The Kernel	118
Building the Kernel	119
The Freestanding Environment	119
Booting the Kernel	120
What's Next	120
Conclusion	121
What's Next	121
Thank You	121
Appendixes	122
Appendix A: Talking to a Real Disk	123
The ATA Controller	123
Port I/O	123
Issuing a Command	123
Opening	124
Reading	124
Writing	124
Completing the Block Device	125
Replacing the RAM Disk	125
Building and Running	125
Appendix B: Long Filename Support (LFN / VFAT)	126
Finding the Fragments	126
Anatomy of an LFN Entry	126
A Quick Look at UTF-16	126
Decoding the Fragments	126

Reassembling the Name	127
The Checksum	127
What Full LFN Support Would Take	127
Appendix C: The Boot Sector's Leftovers	129
Hidden Sectors (Partitioning)	129
CHS (Cylinder-Head-Sector) Addressing	129
Media Descriptor	129
Extended BPB – Additional Fields	129
OEM Name (Original Equipment Manufacturer)	129
Booting from FAT12	129
Field Reference	130
Appendix D: FAT Family Reference	131
FAT Entry Widths and Values	131
Root directory	131
BPB	131
Volume Size Limits	131
Further Reading	131

Introduction

You want to understand how filesystems really work: not how to call `open()`, but what the bytes on disk actually mean. FAT12 is the one to do it with – simple enough to hold entirely in your head, real enough to run on bare metal. Over twelve chapters we will implement every operation ourselves in C (read, write, create directories, delete, move, and format), then boot the exact same code inside our own kernel:

```
FAT12 kernel booted
Opening block device ..... OK
Formatting volume ..... OK
Mounting volume ..... OK
Writing /HELLO.TXT ..... OK
Listing / ..... OK

    FAT12VOL
    HELLO.TXT

Reading /HELLO.TXT ..... OK

    Hello, FAT12!

Unmounting volume ..... OK
Closing block device ..... OK
```

Figure 1. Where we end up in Chapter 12: our own kernel, booted in QEMU, formatting a FAT12 volume, writing a file, listing it, and reading it back.

No operating system underneath, no filesystem code we did not write. Just an emulated PC, 4 MiB of RAM standing in for a disk, and code we typed ourselves. (Appendix A swaps that RAM disk for an ATA driver.)

Why FAT12?

Almost every PC sold in the last decade boots by reading a FAT volume: UEFI firmware loads its bootloader from the EFI System Partition, and the UEFI

specification requires that partition to be FAT. USB sticks, SD cards up to 32 GB, the floppy images so many OSDev tutorials boot from: FAT again.

This book focuses on FAT12, the variant DOS started with. Understand it and you already understand most of FAT16 and FAT32 too – the core concepts transfer directly. And FAT12 strips everything to the essentials: a single table links every file's clusters together (no extents, no block groups, no B-trees, just a linked list), and the root directory sits at a fixed position with a fixed size, so finding / takes no chain-walking at all. The volumes we work with are just a few MiB, small enough to hexdump every structure on them end to end.

Why Not Just Generate It?

An AI assistant will write you a FAT12 implementation in minutes. It will compile. It might even list a directory. Whether it packs the 12-bit FAT entries correctly is another matter, and a bug there does not crash. It quietly overwrites four bits of the neighboring entry, breaking some other file's cluster chain, and you find out much later when that file comes back as garbage. And no amount of reading the code will catch it unless you know exactly where those twelve bits belong.

This book gives you that understanding, and the tools to prove it. We decode every structure from a real volume created by `mkfs.fat`, and verify our work at every stage: hexdump the disk image, trace what the code did, and compare the two. And when we start writing, we hand the result to Linux's own FAT driver, which never saw our code. Once you know what the bytes *should* be, generated code stops being something you have to trust and becomes something you can review.

Who This Book Is For

You have a kernel that boots and prints to the screen, and now you want it to load files. Every program you want to run is sitting in a filesystem your code cannot parse yet, and FAT12 is the shortest path from here to there.

You do not need to have gotten that far, either. Everything up to the final chapter runs as a command-line tool on your own machine, so the kernel is where we end up rather than where you have to start. The only thing assumed

is C: pointers, structs, and bitwise operations. The main chapters need no assembly beyond a 27-line Multiboot stub in Chapter 12.

If you have tried this before, you know the existing material: wiki articles and tutorials that stop once a file can be read, or switch to assembly halfway through, and a spec that gives you the formulas but not how they fit together into working code. This book goes the whole way, from the first sector read to formatting a fresh volume, all in C.

It is not a C tutorial, and it implements FAT12 only. FAT16 and FAT32 are close cousins (Appendix D shows exactly how little changes), and long filenames get an appendix of their own rather than full support.

What We Will Build

The same FAT12 code runs both as a CLI tool and inside our kernel because it never touches storage directly. Every sector it reads or writes goes through a small BlockDevice interface, and each platform plugs in its own backend: a disk image file for the CLI tool we test with as we go, a RAM disk for the kernel, and in Appendix A a real ATA disk driver. Three very different backends, one FAT12 implementation that never knows the difference.

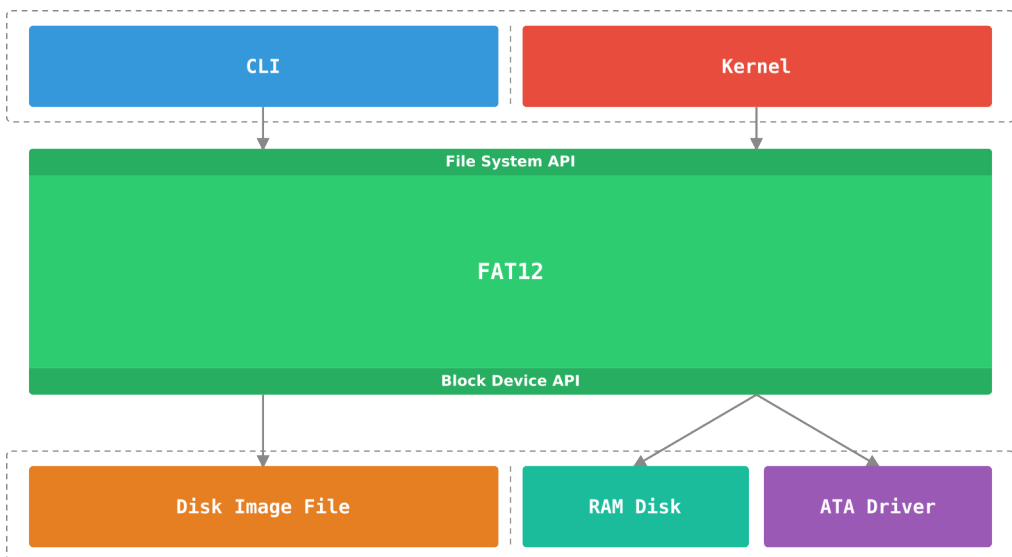


Figure 2. Architecture overview: the CLI tool and the kernel call FAT12 through the File System API, and FAT12 reaches storage through the Block Device API.

How This Book Is Structured

The book is split into three parts, plus a set of appendixes:

Part I: Reading – From raw bytes to file contents. We set up the toolchain, build a portable `BlockDevice` abstraction, and decode the boot sector into the geometry that powers the entire filesystem. Then we walk the root directory and read our first file straight off the volume. To read files of any size, we take apart the File Allocation Table and its famously awkward 12-bit entries in Chapter 5 (the part most tutorials wave at instead of explaining), and finally we follow any path down into nested subdirectories.

Part II: Writing – Write files, create directories, delete, and move. Every change touches more than one place on the volume, so the order of our writes decides what a crash can break. Delete a file in the wrong order and a badly timed crash leaves its entry pointing at clusters the FAT already calls free; the next file to claim them is cross-linked with it, and the two silently corrupt each other. Delete it in the right order and the worst a crash can do is leak a little free space.

Part III: Advanced – Everything so far assumed someone else formatted the volume for us. Now we write our own formatter and create a FAT12 volume from nothing. Then comes the payoff: we compile our FAT12 code into a kernel, boot it in QEMU, and watch it format, write, and read its own RAM disk.

Appendixes – Topics I pulled out of the chapters' straight line but did not want to cut. One swaps the kernel's RAM disk for an ATA driver so what the kernel writes lands on a disk image we can inspect afterward, another introduces long filenames (LFN/VFAT), a third catalogs the boot sector fields we never touched, and a fourth lines FAT12 up against its FAT16 and FAT32 cousins.

How to Read This Book

Read the chapters front to back – each one builds on the last – then browse the appendixes in whatever order interests you. If you want one chapter to judge the book by, make it Chapter 5. Code appears with the explanation: type it, run it, hexdump it, break it. Every chapter ends with code that compiles with zero warnings under `-Wall -Wextra` on a current GCC, and a command we run to

watch it work. If a build fails or a hexdump does not match, the finished code for every chapter is on GitHub as a safety net (the Prerequisites show how to get it).

A few conventions keep the listings easy to follow:

- **Every listing is headed by its file path**, relative to the project root, so you always know which file to create or edit.
- `/* --- NEW --- */` marks lines added to a function or struct we already wrote in an earlier chapter. Update the existing definition rather than typing a second copy.
- `<snake_case>` in a command is a value you supply, and the sentence right before the listing tells you what to put there.
- **Production Note** callouts mark where we took a shortcut that a production implementation would not.

Not Production Code

We skip some checks on purpose, but only ones that have nothing to do with FAT12 itself, such as memory allocation failures and general I/O errors from the block device. Everything that belongs to FAT12 stays in: path validation, attribute checks, bounds on cluster chains, cycle detection, and write ordering that keeps a crash from corrupting more than it has to. Wherever a simplification would matter in production, a Production Note says so.

About the Author

I am Björn Götz, a software developer from a small village in Hesse, Germany. I taught myself to program at twelve, started a three-year software developer apprenticeship at sixteen, and have been building software professionally ever since, mostly backends, plus full-stack web and mobile apps, including the cloud infrastructure they run on. After hours, I go the other direction: operating systems, assembly, and emulators (I wrote my own CHIP-8 emulator just to see how the magic works).

This book started out far bigger. First it was going to cover the most common filesystems, then just the whole FAT family, and each time the scope

buried the part worth learning. So I kept cutting until one filesystem was left, small enough to build from scratch and understand properly. I could not find a book that builds FAT12 step by step in C. The spec tells you what every field means, in roughly the tone of a tax form; building the thing yourself is where it actually sticks, and where it gets fun. So I wrote that book.

Feedback

Found an error? Have a question? Email me at bjoern.goetz2001@gmx.de or open an issue on the book's GitHub repository at github.com/SoftwareFreak1/FAT12. If you enjoyed the book, I would appreciate a review wherever you bought it.

Prerequisites

Before we write any code, let's set up the environment. We do this once, and it covers everything we need for the rest of the book.

Operating System and CPU Architecture

Every command and instruction in this book targets x86 Ubuntu. Don't worry if your machine runs Windows, macOS, or has a non-x86 CPU – spin up a virtual machine (VirtualBox¹ is a solid free option), or skip the VM entirely and follow the Docker instructions below.

x86 Ubuntu in Docker

Install Docker by following the official guide². Once Docker is running, start a container with the command below. Replace `<host_path>` with the directory on your machine where you want the book's code to live. Inside the container, it shows up as `/workspace`, and everything we write there is stored in `<host_path>` on your machine, so it stays safe even if the container is deleted:

```
1 docker run \  
2   -it \  
3   --name fat12 \  
4   --privileged \  
5   --platform linux/amd64 \  
6   -v <host_path>:/workspace \  
7   -w /workspace \  
8   ubuntu \  
9   bash
```

¹<https://www.virtualbox.org/>

²<https://docs.docker.com/get-docker/>

The `--privileged` flag is required for loop device mounts, which we use throughout the book, and `--platform linux/amd64` always gives us x86 Ubuntu (native on x86 machines, emulated everywhere else).

The container keeps everything we install, so this setup only happens once. Exiting the shell just stops the container. To pick up where we left off in a later session, restart it with the command below instead of running `docker run` again:

```
1 docker start -ai fat12
```

The container shell runs as root, and the Ubuntu image doesn't even include `sudo`. Many commands in this book start with `sudo` because native Ubuntu and VM readers need it, so we install it once here. That way, every command works exactly as shown, with no need to strip `sudo` each time:

```
1 apt update
2 apt install sudo
```

From there, run the commands in the following sections to set up the tools one by one.

Command-Line Tools

We build all the code we write with the GCC toolchain. We also need `dosfstools`, which provides the `mkfs.fat` command we use to create FAT disk images until we can create our own, and `xxd`, which we use throughout the book to hexdump disk structures.

Install all three in one shot:

```
1 sudo apt update
2 sudo apt install build-essential dosfstools xxd
```

Kernel Tools (Chapter 12 Only)

Chapter 12 boots our code inside a 32-bit i386 kernel, which needs two extras: 32-bit headers to compile it and QEMU to run it. Feel free to skip both for now and come back once you reach Chapter 12.

First, the 32-bit headers we need to compile the kernel:

```
1 sudo apt update
2 sudo apt install libc6-dev-i386
```

Running it needs the x86 system emulator from QEMU. If you are using a VM or Docker, install QEMU directly on your machine, not inside the VM or container. Follow the official install guide for your platform³.

Getting the Code (Optional)

There is nothing to clone before we start: every chapter shows every line of code we type. Still, the finished code⁴ makes a great safety net. When a build fails or a hexdump doesn't match the book, you can compare your code against a known-good version. To get it, clone the repository:

```
1 git clone https://github.com/SoftwareFreak1/FAT12
```

Each chapter has its own directory (chapter-01 through chapter-12) holding the code exactly as it stands at the end of that chapter, so you can check your work or jump in at any point. Appendixes A and B add code too, so they get their own directories as well (appendix-a, appendix-b).

³<https://www.qemu.org/download/>

⁴<https://github.com/SoftwareFreak1/FAT12>

Part I: Reading

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Chapter 1: Hello, Disk!

The storage a device exposes is really just a long sequence of ones and zeros, with no structure or organization of its own. A filesystem takes that raw sequence and organizes it into files and directories – simple abstractions that hide all the space management happening underneath. Before we can build any of that, we need some storage, and a device-independent way to interact with it from our filesystem code. By the end of this chapter, we will have both.

Creating a Disk Image

We need some storage to experiment on, but not real hardware. A file works just as well – safer to play with, faster to create and discard, and if we wipe it by accident, nothing important is lost.

Luckily, creating one is straightforward on Linux. We first create an empty 4 MiB file to act as our disk, then use `mkfs.fat` (the standard tool for formatting FAT volumes) to write a FAT12 filesystem with the volume label `MYDISK` onto it (we will also set some geometry parameters here that we will explain throughout the book). Put it all in a script called `create_disk.sh` so we can create a fresh disk image whenever we need one:

`create_disk.sh`

```
1  #!/bin/bash
2  dd if=/dev/zero of=disk.img bs=1M count=4
3  mkfs.fat -F 12 -s 2 -S 512 -n MYDISK disk.img
```

We first need to make the script executable, then run it:

```
1  chmod +x create_disk.sh
2  ./create_disk.sh
```

We should now have `disk.img` – a 4 MiB disk image containing a valid FAT12 filesystem. Running the command below gives us a peek at the first few bytes of what `mkfs.fat` actually wrote:

```
1 xxd -g 1 disk.img | head -6

1 00000000: eb 3c 90 6d 6b 66 73 2e 66 61 74 00 02 02 01 00  .<.mkfs.fat.....
2 00000010: 02 00 02 00 20 f8 0c 00 20 00 02 00 00 00 00 00  .... .
3 00000020: 00 00 00 00 80 00 29 91 45 15 a6 4d 59 44 49 53  ....).E..MYDIS
4 00000030: 4b 20 20 20 20 20 46 41 54 31 32 20 20 20 0e 1f  K    FAT12  ..
5 00000040: be 5b 7c ac 22 c0 74 0b 56 b4 0e bb 07 00 cd 10  .[|."t.V.....
6 00000050: 5e eb f0 32 e4 cd 16 cd 19 eb fe 54 68 69 73 20  ^..2.....This
```

We can already see the volume label MYDISK inside it, clearly visible in the ASCII column at offset 0x002b.

Sectors and LBA

A storage device does not transfer data one byte at a time – it reads and writes in fixed-size blocks called **sectors**. The number of bytes in each sector is device-dependent, but always a power of 2. We call this the **sector size**.

We passed -S 512 to mkfs.fat when formatting our disk – this set the sector size to 512 bytes. The value is recorded on the volume and must match what the device itself reports for the math to work. We will see in Chapter 2 exactly how the sector size is stored on the volume.

Every sector also has an address. We number them sequentially, starting from 0 – a scheme called Logical Block Addressing (LBA). Sector 0 is the very first block on the storage device, sector 1 is the next, all the way to the end. You can see this in the image below – the LBA is shown inside each box, and the byte offset is shown at the bottom.

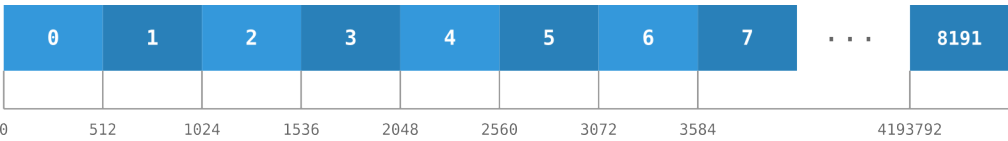


Figure 3. LBA addressing – sectors as numbered blocks in a linear array

Our 4 MiB disk image, with 512-byte sectors, has 8192 of them (0 through 8191):

```
1 Total sectors = Storage device size (bytes) / Sector size (bytes)
2               = 4,194,304 / 512
3               = 8192
```



Why sector size has to match: The sector size baked into the volume's own layout and the sector size the device reports are two separate numbers – nothing forces them to agree. Every LBA our code computes assumes one specific number of bytes per sector; whatever reads the storage device turns that same LBA into a physical byte offset using its own idea of sector size. If the two disagree, the same LBA points to different bytes on each side, silently corrupting the storage device.

The Block Device

Filesystem code should never talk to storage hardware directly. Instead, it defines an abstract interface – a Block Device API – that each storage driver implements. The filesystem reads and writes through that interface, never knowing or caring whether the underlying device is an ATA hard disk, an NVMe SSD, or a file on another filesystem. This is exactly what lets us swap the block device implementation without touching a single line of filesystem code – first in Chapter 12, when we move our implementation into a kernel backed by a RAM disk, then again in Appendix A, when that RAM disk gets swapped for an ATA driver.



It is called a *block* device because the storage device deals in fixed-size blocks; the naming is a bit confusing since on FAT systems we call them *sectors* rather than blocks, but “block device” is the established term across operating systems and beyond.

We start by defining the interface every block device must implement:

`include/block_device.h`

```

1  #ifndef BLOCK_DEVICE_H
2  #define BLOCK_DEVICE_H
3
4  #include <stdint.h>
5
6  typedef struct BlockDevice BlockDevice;
7
8  int block_device_read(
9      BlockDevice* device,
10     uint64_t lba,
11     uint32_t sector_count,
12     void* buffer
13 );
14
15 int block_device_write(
16     BlockDevice* device,
17     uint64_t lba,
18     uint32_t sector_count,
19     const void* buffer
20 );
21
22 uint32_t block_device_sector_size(BlockDevice* device);
23 uint64_t block_device_sector_count(BlockDevice* device);
24 int block_device_close(BlockDevice* device);
25
26 #endif

```

Five operations make up the interface. `block_device_read` and `block_device_write` move a given number of sectors, starting at a given LBA, between the block device and a caller-supplied buffer. `block_device_sector_size` and `block_device_sector_count` report the device's geometry – the fixed number of bytes per sector and the total number of sectors on the device. `block_device_close` releases whatever resources the implementation is holding.



These functions are plain global symbols, not function pointers in a vtable. If we needed to support multiple devices at once (say, a SATA disk and a USB stick), each `BlockDevice` instance would need its own function table. This book keeps it simple: one set of definitions per platform, and the linker picks whichever `.c` file we compile.

Production Note: Every I/O operation can, and eventually will, fail – but “0 returned” and “durable on disk” are two different guarantees. `block_device_read`, `block_device_write`, and `block_device_close` all return an `int` status code, and we ignore it throughout this book, for the same reason we ignore `malloc` failures elsewhere: checking every call would bury the FAT12 logic under `if` statements for operations we call constantly. But even a checked 0 from `block_device_write` only means the bytes were handed off through the interface, not that they have reached the physical disk yet – real storage stacks buffer writes at multiple layers (the C library, the OS page cache, the drive’s own cache) and flush them later, with explicit flush points marking where data must actually be durable. Keep this distinction in mind once we get to write ordering later in the book – what we can reason about is the order our `BlockDevice` calls happen in, not when the bytes actually land on a real disk.

You may have noticed something missing from the header above: `block_device_close`, but no matching `open`. That is not an oversight – opening a device needs arguments specific to whatever it actually is, so it cannot be part of the common interface. `Open` is the one place in this whole design where we are allowed to know exactly which implementation we are dealing with; every other operation stays behind the abstraction.

Debug Logging

Before we implement our first block device, let’s add a little helper macro for printing debug output. We place debug prints throughout the book to trace the operations and translations our code performs, giving us insight into what is actually happening under the hood and a big help when tracking down bugs:

`include/debug.h`

```
1  #ifndef DEBUG_H
2  #define DEBUG_H
3
4  #if DEBUG
5  #include <stdio.h>
6
7  #define DBG_PRINT(...) \
8      do { \
9          printf("\033[36m"); \
10         printf(__VA_ARGS__); \
11         printf("\033[0m"); \
12     } while (0)
13 #else
14 #define DBG_PRINT(...)
15 #endif
16
17 #endif
```

DBG_PRINT is defined conditionally, so it can be swapped out for an empty version whenever we don't want debug prints cluttering our output. When enabled, it forwards every argument straight to `printf`, wrapping the message in ANSI escape codes that print it in cyan and make it stand out from our CLI's normal output. Notice the header never defines `DEBUG` itself – we'll set its value later, at compile time.

The File Block Device

To interact with our disk image, we need a block device that reads and writes it. For that, we declare an open function that takes the path to the disk image file as its argument:

platform/cli/file_block_device.h

```

1  #ifndef FILE_BLOCK_DEVICE_H
2  #define FILE_BLOCK_DEVICE_H
3
4  #include "block_device.h"
5
6  BlockDevice* file_block_device_open(const char* path);
7
8  #endif

```

Then implement it:

platform/cli/file_block_device.c

```

1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <inttypes.h>
4  #include "debug.h"
5  #include "block_device.h"
6
7  struct BlockDevice {
8      FILE* file;
9      uint32_t sector_size;
10     uint64_t sector_count;
11 };
12
13 BlockDevice* file_block_device_open(const char* path) {
14     BlockDevice* device = (BlockDevice*)malloc(sizeof(BlockDevice));
15
16     device->file = fopen(path, "r+");
17     if (device->file == NULL) {
18         free(device);
19         return NULL;
20     }
21
22     device->sector_size = 512;
23
24     fseeko(device->file, 0, SEEK_END);
25     device->sector_count = (uint64_t)(ftello(device->file) /
26     ↪ device->sector_size);
27
28     return device;
29 }

```

We first need to define what the `BlockDevice` struct actually holds: a file handle for the open disk image, the sector size we use, and the sector count. `file_block_device_open` allocates a new `BlockDevice` and fills in

each of these. It opens the disk image file and checks whether `fopen` actually succeeded – if `disk.img` does not exist (or we lack permission to open it), `fopen` returns `NULL`, and we free the struct we just allocated and return `NULL` ourselves.

We hardcode the sector size to 512, since a raw file has no sector size of its own the way a real device does – but it must match the sector size we used when formatting our disk image. The sector count is computed by seeking to the end of the file to find its size in bytes and dividing by the sector size.

Now we continue with reading and writing:

platform/cli/file_block_device.c

```

1  int block_device_read(
2      BlockDevice* device,
3      uint64_t lba,
4      uint32_t sector_count,
5      void* buffer
6  ) {
7      DBG_PRINT("[ block_device ] read %" PRIu32 " sector(s) starting at LBA %"
8          ↪ PRIu64 "\n", sector_count, lba);
9
10     size_t total_bytes = sector_count * device->sector_size;
11     off_t offset = lba * device->sector_size;
12
13     DBG_PRINT("[ file_io      ] read %zu bytes at 0x%" PRIx64 "\n",
14         ↪ total_bytes, (uint64_t)offset);
15
16     fseeko(device->file, offset, SEEK_SET);
17     fread(buffer, 1, total_bytes, device->file);
18
19     return 0;
20 }
21
22 int block_device_write(
23     BlockDevice* device,
24     uint64_t lba,
25     uint32_t sector_count,
26     const void* buffer
27 ) {
28     DBG_PRINT("[ block_device ] write %" PRIu32 " sector(s) starting at LBA %"
29         ↪ PRIu64 "\n", sector_count, lba);
30
31     size_t total_bytes = sector_count * device->sector_size;
32     off_t offset = lba * device->sector_size;

```

```

31     DBG_PRINT("[ file_io      ] write %zu bytes at 0x%" PRIx64 "\n",
    ↪     total_bytes, (uint64_t)offset);
32
33     fseeko(device->file, offset, SEEK_SET);
34     fwrite(buffer, 1, total_bytes, device->file);
35
36     return 0;
37 }

```

Each function translates a request expressed in LBA and sector count into a file operation expressed in byte offset and byte count. The LBA is translated into a byte offset by multiplying it with the sector size ($\text{Byte Offset} = \text{LBA} * \text{Sector Size}$). The sector count is translated the same way, into a byte count ($\text{Byte Count} = \text{Sector Count} * \text{Sector Size}$). That calculated byte offset and byte count are then used to read or write the disk image file.

The remaining three operations complete the interface:

platform/cli/file_block_device.c

```

1  int block_device_close(BlockDevice* device) {
2      fclose(device->file);
3      free(device);
4
5      return 0;
6  }
7
8  uint32_t block_device_sector_size(BlockDevice* device) {
9      return device->sector_size;
10 }
11
12 uint64_t block_device_sector_count(BlockDevice* device) {
13     return device->sector_count;
14 }

```

`block_device_close` closes the file and frees the memory of the `BlockDevice` struct. `block_device_sector_size` and `block_device_sector_count` just return the values we already set on open.

Production Note: `fopen` is the one file operation we do check, because its failure blocks everything downstream – without an open file, there is nothing

for any other operation to read, write, or close, so `file_block_device_open` returns `NULL` before any of them get a chance to run. Every other file operation here goes unchecked, and for good reason: our FAT12 code ignores the `BlockDevice` interface's own return value anyway, so checking `fseeko`, `fread`, `fwrite`, or `fclose` here would only add complexity with nothing to gain from it. A production driver would check each one and propagate real failures back through the interface instead.

Putting the Block Device to Work

We have a block device and a disk image to point it at. Let's exercise the whole interface: print its sector size and sector count, read the first sector to extract the volume label, then write a new one back.

platform/cli/main.c

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <stdint.h>
4  #include <inttypes.h>
5  #include "block_device.h"
6  #include "file_block_device.h"
7
8  /* same offset as in the xxd dump earlier; Chapter 2 explains why */
9  #define VOLUME_LABEL_OFFSET 0x2b
10
11 int main(void) {
12     /* Hardcoded so every command stays short */
13     BlockDevice* device = file_block_device_open("disk.img");
14     if (device == NULL) {
15         fprintf(stderr, "error: could not open disk.img\n");
16         return 1;
17     }
18
19     uint32_t sector_size = block_device_sector_size(device);
20     printf("Sector size (bytes): %u\n", sector_size);
21     printf("Sector count: %" PRIu64 "\n", block_device_sector_count(device));
22
23     uint8_t sector[sector_size];
24     block_device_read(device, 0, 1, sector);
25     printf("Volume label: %.11s\n", (const char*)&sector[VOLUME_LABEL_OFFSET]);

```

```

26
27     memcpy(&sector[VOLUME_LABEL_OFFSET], "HELLO DISK!", 11);
28     block_device_write(device, 0, 1, sector);
29
30     block_device_close(device);
31
32     return 0;
33 }

```



The label field is exactly 11 bytes, not null-terminated. `%.11s` prints exactly 11 characters without needing a terminator.

Before we can actually run it, let's add one last script that discovers every source file automatically and compiles them into one executable, which we will use throughout the rest of this book. It also reads a `DEBUG` environment variable and passes it straight to the compiler as a macro, so we can flip debug tracing on for a single build:

build.sh

```

1  #!/bin/bash
2  SRC=""
3  for d in src platform/cli; do
4      for f in "$d"/*.c; do
5          [ -f "$f" ] && SRC="$SRC $f"
6      done
7  done
8  gcc -Wall -Wextra -DDEBUG=${DEBUG:-0} -iquote include -iquote src -iquote
   → platform/cli -o fat12-cli $SRC

```

We first need to make the script executable, then run it:

```

1  chmod +x build.sh
2  ./build.sh

```

We should now have an executable named `fat12-cli`. Let's run it with:

```

1  ./fat12-cli

```

We should see the following:

```

1 Sector size (bytes): 512
2 Sector count: 8192
3 Volume label: MYDISK

```

The sector count matches what we calculated earlier, and the sector size and volume label match what we set when we created the disk image. Now let's confirm the modified volume label actually landed on our disk image too, using `xxd` to look at it again, the same way we did at the start of the chapter:

```

1 xxd -g 1 disk.img | head -6

```

```

1 00000000: eb 3c 90 6d 6b 66 73 2e 66 61 74 00 02 02 01 00  .<.mkfs.fat.....
2 00000010: 02 00 02 00 20 f8 0c 00 20 00 02 00 00 00 00 00  .... ..
3 00000020: 00 00 00 00 80 00 29 91 45 15 a6 48 45 4c 4c 4f  .....).E..HELLO
4 00000030: 20 44 49 53 4b 21 46 41 54 31 32 20 20 20 0e 1f  DISK!FAT12 ..
5 00000040: be 5b 7c ac 22 c0 74 0b 56 b4 0e bb 07 00 cd 10  .[|."t.V.....
6 00000050: 5e eb f0 32 e4 cd 16 cd 19 eb fe 54 68 69 73 20  ^..2.....This

```

HELLO DISK! sits exactly where MYDISK used to, at offset `0x002b` – only those 11 bytes changed.

Tracing the Translation

Let's look at what's happening under the hood. We create a fresh disk image and run the same code again, this time with debug output enabled:

```

1 ./create_disk.sh
2 DEBUG=1 ./build.sh
3 ./fat12-cli

```

```
1 Sector size (bytes): 512
2 Sector count: 8192
3 [ block_device ] read 1 sector(s) starting at LBA 0
4 [ file_io       ] read 512 bytes at 0x0
5 Volume label: MYDISK
6 [ block_device ] write 1 sector(s) starting at LBA 0
7 [ file_io       ] write 512 bytes at 0x0
```

Now we can see each read and write request the block device receives, alongside the byte offset and byte count our file block device translated it into.



Keep this trick in mind for the rest of the book: any time something looks off – an address translation error, a read that returns garbage – rebuild with `DEBUG=1 ./build.sh` and the full read/write path prints to the terminal. As we add more functionality in future chapters, we will also add more debug calls so the trace builds up.

What's Next

This chapter laid the foundation everything that follows depends on – a disk image with a valid FAT12 filesystem on it, and a device-independent way to talk to it from our filesystem code. Chapter 2 puts that foundation to work, introducing the boot sector – the core every filesystem operation needs.

Chapter 2: Mounting the Filesystem

Chapter 1 laid the foundation: a block device that can read and write raw sectors. But raw sectors alone don't know anything about the filesystem sitting on top of them – before any operation can work, we first need to mount it. Mounting reads the volume's metadata once, sets up the internal state every later operation relies on, and hands back a handle we pass to everything from here on. By the end of this chapter, we will have a mounted filesystem with every one of its parameters printed out in front of us, ready to build later operations on.

Meet the Boot Sector

The boot sector is always the first sector (LBA 0) of every FAT volume and exactly one sector long. It holds every parameter the FAT format doesn't fix on its own – without it, we don't know anything about the filesystem. It is a set of fields laid out at fixed positions inside it. Here is what it looks like:

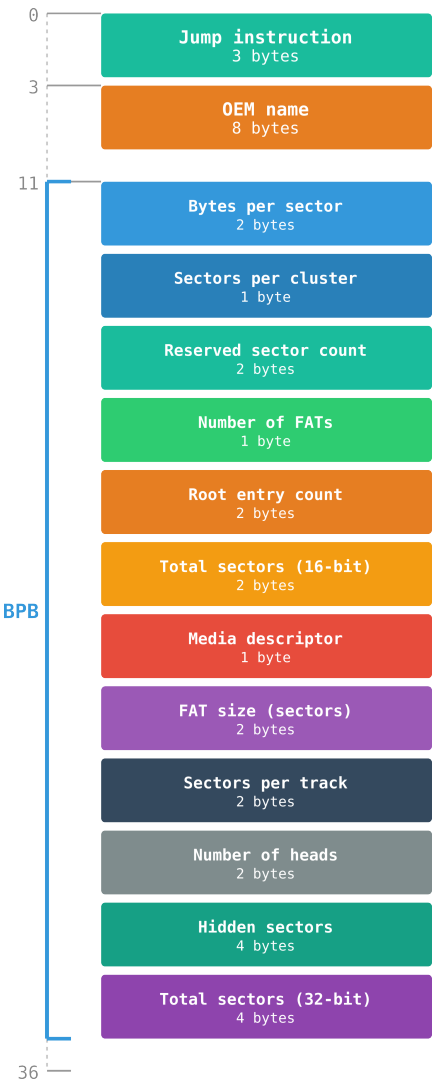


Figure 4. Boot sector layout, part 1

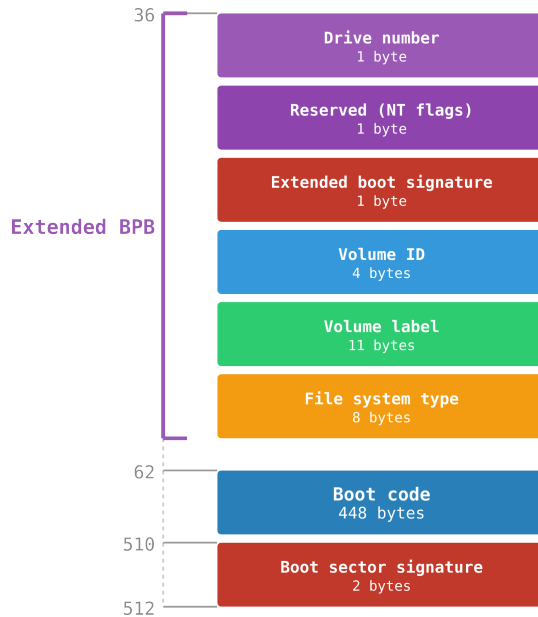


Figure 5. Boot sector layout, part 2



The boot sector's defined layout adds up to exactly 512 bytes, matching our disk image's sector size. A sector size larger than 512 is valid too – and on a storage device like that the layout does not move, every field lands at the same offset it does here. The extra bytes at the end aren't used by FAT itself.

Each field's meaning gets explained the moment we actually need it – spread out across the chapters ahead, not dumped here all at once. A handful of unused, legacy fields skip that treatment entirely and live in Appendix C instead, so this chapter doesn't turn into a long reference that breaks the narrative.

Two regions of it are further grouped into named blocks. The **BIOS Parameter Block (BPB)** is the centerpiece – it holds everything we need to locate the rest of the volume. The **Extended BIOS Parameter Block (EBPB)**, added in later FAT revisions, carries additional identification.



The “BIOS” in BIOS Parameter Block isn't the machine's firmware – the PC ROM BIOS never reads the BPB. It's named after the DOS-internal “DOS BIOS” I/O component, which described the disk to the DOS kernel via this exact block – the name just stuck as a DOS implementation detail, not a firmware reference.

We need an easy, structured way to access each field. For that, we define structs that directly mirror the boot sector's on-disk layout in C.

src/layout.h

```

1  #ifndef LAYOUT_H
2  #define LAYOUT_H
3
4  #include <stdint.h>
5
6  #pragma pack(push, 1)
7
8  typedef struct {
9      uint16_t bytes_per_sector;
10     uint8_t sectors_per_cluster;
11     uint16_t reserved_sector_count;
12     uint8_t num_fats;
13     uint16_t root_entry_count;
14     uint16_t total_sectors_16;
15     uint8_t media;
16     uint16_t fat_size_16;
17     uint16_t sectors_per_track;
18     uint16_t number_of_heads;
19     uint32_t hidden_sectors;
20     uint32_t total_sectors_32;
21 } BPB;
22
23 typedef struct {
24     uint8_t drive_number;
25     uint8_t reserved_nt;
26     uint8_t boot_signature;
27     uint32_t volume_id;
28     char volume_label[11];
29     char file_system_type[8];
30 } ExtendedBPB;
31
32 typedef struct {
33     uint8_t jump[3];
34     char oem_name[8];
35     BPB bpb;
36     ExtendedBPB extended_bpb;
37     uint8_t boot_code[448];
38     uint16_t signature;
39 } BootSector;
40
41 #pragma pack(pop)
42
43 #endif

```

Memory Alignment: When the compiler lays out a struct in memory it normally adds padding between members to satisfy alignment rules – a `uint16_t` after a `uint8_t` might get shifted to an even address. That would break our mapping, because the on-disk layout has no padding. `#pragma pack(push, 1)` tells the compiler to pack every field at the next available byte with no gaps. Notice we push before the first struct and pop after the last – the push/pop pair ensures the packing only applies to our on-disk structs and does not affect any other structs after the pop.

Reading the Boot Sector

Now that we have the boot sector struct in place, we can actually load it using our block device.

src/fat12.c

```

1  #include <stdlib.h>
2  #include <string.h>
3  #include "block_device.h"
4  #include "debug.h"
5  #include "layout.h"
6  #include "fat12.h"
7
8  static BootSector read_boot_sector(BlockDevice* device)
9  {
10     uint32_t sector_size = block_device_sector_size(device);
11     void* buffer = malloc(sector_size);
12     block_device_read(device, 0, 1, buffer);
13
14     BootSector result;
15     memcpy(&result, buffer, sizeof(BootSector));
16     free(buffer);
17
18     return result;
19 }
```

`read_boot_sector` allocates a buffer the size of one sector, reads the first sector (LBA 0) into it, then copies it byte for byte directly into our `BootSector`

struct – which works because the struct is already packed to match the boot sector’s exact on-disk layout.



Endianness: FAT12 stores every multi-byte value in **little-endian** (least significant byte first). Copying raw disk bytes into our struct is endian-agnostic – every CPU places them in the same memory order. Reading them as a number is not: take `bytes_per_sector`, stored on disk as the two bytes `0x00 0x02`. x86 treats the lower-addressed byte as least significant, giving `0x0200 = 512`, the correct value. A big-endian CPU (some embedded PowerPC or MIPS) would read the same two bytes the other way around, giving `0x0002 = 2` – same bytes, wrong number. Throughout this book, whenever we read a value wider than one byte, we are relying on x86’s endianness matching FAT’s little-endian on-disk format. The pattern for a big-endian CPU is always the same: grab the bytes in the little-endian order they appear on disk and convert to the CPU’s native byte order before use, and reverse them again before writing back.

Putting It All Together

Everything we need is in place – the boot sector struct and the code to read it. Now we can implement mounting (and unmounting).

First, let’s define the two operations our public file system API exposes – `fat12_mount` and `fat12_umount` – along with the opaque `FAT12FS` handle they hand back and forth.

`include/fat12.h`

```

1  #ifndef FAT12_H
2  #define FAT12_H
3  #include <stdint.h>
4  #include "block_device.h"
5
6  typedef struct FAT12FS FAT12FS;
7
8  FAT12FS* fat12_mount(BlockDevice* device);
9  void fat12_umount(FAT12FS* fs);
10
11 #endif

```

Production Note: Unix-like kernels mount a volume at a path in a global VFS (Virtual File System) tree, so many volumes can coexist under one namespace. Our code manages exactly one volume, so we skip the mount point entirely.

Now let's define what FAT12FS actually holds, and implement both functions.

src/fat12.c

```

1  struct FAT12FS {
2      BlockDevice* device;
3      BootSector bs;
4  };
5
6  FAT12FS* fat12_mount(BlockDevice* device)
7  {
8      DBG_PRINT("[ fat12          ] --- fat12_mount() ---\n");
9      FAT12FS* fs = malloc(sizeof(FAT12FS));
10     fs->device = device;
11     fs->bs = read_boot_sector(device);
12
13     /* jump, boot_code, reserved_nt: not used by FAT itself, so we skip them */
14     DBG_PRINT("[ fat12          ] OEM Name: %.8s\n", fs->bs.oem_name);
15     DBG_PRINT("[ fat12          ] Bytes Per Sector: %u\n",
16         ↪ fs->bs.bpb.bytes_per_sector);
17     DBG_PRINT("[ fat12          ] Sectors Per Cluster: %u\n",
18         ↪ fs->bs.bpb.sectors_per_cluster);
19     DBG_PRINT("[ fat12          ] Reserved Sector Count: %u\n",
20         ↪ fs->bs.bpb.reserved_sector_count);

```

```

18     DBG_PRINT("[ fat12      ] Number of FATs: %u\n", fs->bs.bpb.num_fats);
19     DBG_PRINT("[ fat12      ] Root Entry Count: %u\n",
    ↪ fs->bs.bpb.root_entry_count);
20     DBG_PRINT("[ fat12      ] Total Sectors (16): %u\n",
    ↪ fs->bs.bpb.total_sectors_16);
21     DBG_PRINT("[ fat12      ] Media Descriptor: 0x%02x\n", fs->bs.bpb.media);
22     DBG_PRINT("[ fat12      ] FAT Size (sectors): %u\n",
    ↪ fs->bs.bpb.fat_size_16);
23     DBG_PRINT("[ fat12      ] Sectors Per Track: %u\n",
    ↪ fs->bs.bpb.sectors_per_track);
24     DBG_PRINT("[ fat12      ] Number of Heads: %u\n",
    ↪ fs->bs.bpb.number_of_heads);
25     DBG_PRINT("[ fat12      ] Hidden Sectors: %u\n",
    ↪ fs->bs.bpb.hidden_sectors);
26     DBG_PRINT("[ fat12      ] Total Sectors (32): %u\n",
    ↪ fs->bs.bpb.total_sectors_32);
27     DBG_PRINT("[ fat12      ] Drive Number: 0x%02x\n",
    ↪ fs->bs.extended_bpb.drive_number);
28     DBG_PRINT("[ fat12      ] Boot Signature: 0x%02x\n",
    ↪ fs->bs.extended_bpb.boot_signature);
29     DBG_PRINT("[ fat12      ] Volume ID: 0x%08x\n",
    ↪ fs->bs.extended_bpb.volume_id);
30     DBG_PRINT("[ fat12      ] Volume Label: %.11s\n",
    ↪ fs->bs.extended_bpb.volume_label);
31     DBG_PRINT("[ fat12      ] File System Type: %.8s\n",
    ↪ fs->bs.extended_bpb.file_system_type);
32     DBG_PRINT("[ fat12      ] Signature: 0x%04x\n", fs->bs.signature);
33
34     return fs;
35 }
36
37 void fat12_umount(FAT12FS* fs)
38 {
39     free(fs);
40 }

```

`fat12_mount` allocates a new `FAT12FS` handle, storing the block device and caching the boot sector inside it so every later operation can reach it without reading it again. `fat12_mount` reads the boot sector once via `read_boot_sector`, debug-prints every field FAT itself actually uses, then returns the handle. `fat12_umount` simply frees the space the handle occupies.

Mounting in Action

Let's put it to use by replacing our `main` from Chapter 1 with the following code – it keeps opening and closing the block device, and mounts and unmounts the

filesystem in between:

platform/cli/main.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "block_device.h"
4  #include "file_block_device.h"
5  #include "fat12.h"
6
7  int main(void) {
8      /* Hardcoded so every command stays short */
9      BlockDevice* device = file_block_device_open("disk.img");
10     if (device == NULL) {
11         fprintf(stderr, "error: could not open disk.img\n");
12         return 1;
13     }
14
15     FAT12FS *fs = fat12_mount(device);
16     fat12_umount(fs);
17
18     block_device_close(device);
19
20     return 0;
21 }
```

Create a fresh disk image, then build and run with debug output enabled:

```
1  ./create_disk.sh
2  DEBUG=1 ./build.sh
3  ./fat12-cli
```

We should see something like:

```

1  [ fat12          ] --- fat12_mount() ---
2  [ block_device ] read 1 sector(s) starting at LBA 0
3  [ file_io       ] read 512 bytes at 0x0
4  [ fat12         ] OEM Name: mkfs.fat
5  [ fat12         ] Bytes Per Sector: 512
6  [ fat12         ] Sectors Per Cluster: 2
7  [ fat12         ] Reserved Sector Count: 1
8  [ fat12         ] Number of FATs: 2
9  [ fat12         ] Root Entry Count: 512
10 [ fat12         ] Total Sectors (16): 8192
11 [ fat12         ] Media Descriptor: 0xf8
12 [ fat12         ] FAT Size (sectors): 12
13 [ fat12         ] Sectors Per Track: 32
14 [ fat12         ] Number of Heads: 2
15 [ fat12         ] Hidden Sectors: 0
16 [ fat12         ] Total Sectors (32): 0
17 [ fat12         ] Drive Number: 0x80
18 [ fat12         ] Boot Signature: 0x29
19 [ fat12         ] Volume ID: 0x72bae6ba
20 [ fat12         ] Volume Label: MYDISK
21 [ fat12         ] File System Type: FAT12
22 [ fat12         ] Signature: 0xaa55

```

Every field should match the output above, except Volume ID – it’s meant to be unique per volume.

We also see the volume label we read in Chapter 1 in the output above. Back then we referenced it by an absolute offset, not knowing why it lived there. Now we know: `volume_label` is a real field in our struct, defining exactly where it fits inside the boot sector and how big it is.

Production Note: We never check Boot Signature before trusting Volume ID, Volume Label, or File System Type – harmless here since we only print them. Appendix C covers why a production driver must check it first.

What’s Next

We can now mount the filesystem: mounting hands back a handle holding the block device and the boot sector – everything every later operation will need. Next, in Chapter 3, we put that handle to its first real use: listing what’s actually in the root directory.

Chapter 3: Listing the Root Directory

Chapter 2 built the base every filesystem operation stands on. But before we can read a single file, we first need to know which files and directories are actually there. By the end of this chapter, we will have extended our filesystem API with a directory iteration API that lists the contents of the root directory (/), used by our own `ls` command.

Locating the Root Directory

Before we can read what's inside the root directory, we first need to know where it lives. For that, let's look at how a FAT12 volume is laid out:

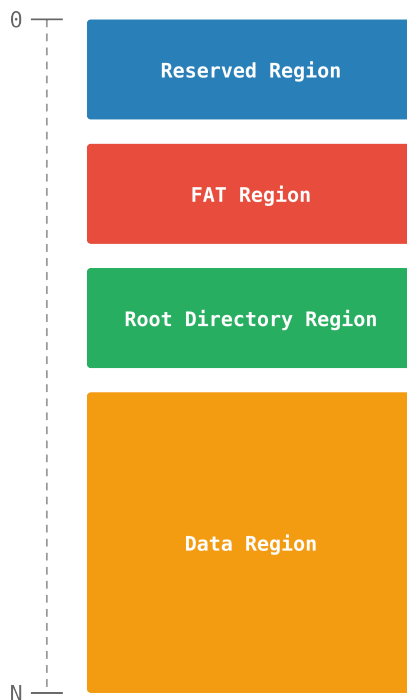


Figure 6. FAT12 volume layout

Every volume splits into four regions, one after another. At the very start sits the reserved region, at LBA 0 – wait, didn't we just say the boot sector lives

at LBA 0, and now the reserved region starts at LBA 0 too? Both are true: the boot sector is the first sector of the reserved region, it just does not have to be the last one (though it usually is). Next comes the FAT region, which we will fully introduce in Chapter 5. After that is what we are actually looking for: the root directory has its own fixed region on the volume. The last region spans everything left over – the data region, introduced in Chapter 4.

Now that we know where the root directory sits, let's actually find it – at which LBA does its region start? At first glance it looks like every region should have a fixed LBA, but only the reserved region does: every other region's start depends on the size of everything before it, and FAT itself does not fix those sizes anywhere. Luckily, the boot sector holds everything we need to work them out, starting with the one fixed point we already know: the reserved region begins at LBA 0. It usually holds nothing but the boot sector, but we cannot assume it is always exactly one sector – its exact size comes from the BPB field `Reserved Sector Count`, which can never be less than 1, since the boot sector alone already accounts for that one. FAT12 itself does not care what, if anything, fills the rest of that space – it only needs to know how many sectors to skip before the FAT region begins.

That puts us at the start of the FAT region. Now the only thing we need to know is how big it is – for that we need two more BPB fields we have not introduced yet: `Number of FATs`, which tells us how many copies of the FAT exist (more on that in Chapter 5), and `FAT Size (sectors)`, which tells us how big each copy is. So calculating the size of the FAT region is simply `Number of FATs * FAT Size (sectors)`. By adding that size to the start LBA, we get the start of the root directory – the formulas below put it all together:

```
1 Reserved region start LBA = 0
2 FAT region start LBA      = Reserved region start LBA + Reserved Sector Count
3 FAT region size (sectors) = Number of FATs * FAT Size (sectors)
4 Root dir start LBA       = FAT region start LBA + FAT region size (sectors)
```

Now that we know where it starts, we also need to know how big it is so we can load it into memory. The root directory itself is nothing more than a sequence of directory entries, each always 32 bytes in size. The BPB's `Root Entry Count` field tells us how many entries it has space for, so we get the total byte size by multiplying it by the size of each entry. Every region on the volume must be allocated in whole sectors, so we round up to the next whole sector. The BPB field `Bytes Per Sector` gives us exactly what we need for

that: the volume's sector size, the same value our block device already reports, now read straight from the BPB's own copy of it – the formula below puts it all together:

```
1 Directory Entry Size          = 32
2 Root directory size (sectors) = (Root Entry Count * Directory Entry Size +
  ↳ Bytes Per Sector - 1) / Bytes Per Sector
```

The `+ Bytes Per Sector - 1` is the standard trick for rounding up with integer math: it pushes any leftover bytes into one more whole sector before the division truncates.



If the block device already knows its own sector size, why does the volume need to store it too? In the DOS era, disk controllers had no self-describing interface of their own – drivers configured the controller *from* the BPB rather than asking it. The BPB kept the volume self-describing, independent of whatever hardware happened to be reading it, so a floppy formatted on one machine still carried its own geometry when read on another.

Unlike subdirectories (which we create in Chapter 8), the root directory is a fixed size. The BPB field `Root Entry Count` tells us how many entries fit. If we fill it up, no new entries can be created in the root – even if there is free space on the disk. The root directory is the one place on a FAT12 volume where the entry count is the bottleneck.

That's a lot of math to take in at once. Take a moment, reread the formulas above until they click, and trace them back against the volume layout diagram – watching how each region's start and size feeds into the next one is what makes the whole chain make sense. We compute each of these values once and cache them inside our mount struct, alongside everything Chapter 2 already stored there – we will need them again and again throughout the book, and each one depends on the one before it. Let's translate it into code:

src/fat12.c

```

1  /* --- NEW --- */
2  #define DIRECTORY_ENTRY_SIZE 32
3
4  // ...
5
6  struct FAT12FS {
7      BlockDevice* device;
8      BootSector bs;
9
10     /* --- NEW --- */
11     uint32_t fat_lba;
12     uint32_t fat_sectors;
13     uint32_t root_dir_lba;
14     uint32_t root_dir_sectors;
15     /* --- END NEW --- */
16 };
17
18 // ...
19
20 FAT12FS* fat12_mount(BlockDevice* device)
21 {
22     DBG_PRINT("[ fat12          ] --- fat12_mount() ---\n");
23     FAT12FS* fs = malloc(sizeof(FAT12FS));
24     fs->device = device;
25     fs->bs = read_boot_sector(device);
26
27     /* jump, boot_code, reserved_nt: not used by FAT itself, so we skip them */
28     DBG_PRINT("[ fat12          ] OEM Name: %.8s\n", fs->bs.oem_name);
29     DBG_PRINT("[ fat12          ] Bytes Per Sector: %u\n",
30         ↪ fs->bs.bpb.bytes_per_sector);
31     DBG_PRINT("[ fat12          ] Sectors Per Cluster: %u\n",
32         ↪ fs->bs.bpb.sectors_per_cluster);
33     DBG_PRINT("[ fat12          ] Reserved Sector Count: %u\n",
34         ↪ fs->bs.bpb.reserved_sector_count);
35     DBG_PRINT("[ fat12          ] Number of FATs: %u\n", fs->bs.bpb.num_fats);
36     DBG_PRINT("[ fat12          ] Root Entry Count: %u\n",
37         ↪ fs->bs.bpb.root_entry_count);
38     DBG_PRINT("[ fat12          ] Total Sectors (16): %u\n",
39         ↪ fs->bs.bpb.total_sectors_16);
40     DBG_PRINT("[ fat12          ] Media Descriptor: 0x%02x\n", fs->bs.bpb.media);
41     DBG_PRINT("[ fat12          ] FAT Size (sectors): %u\n",
42         ↪ fs->bs.bpb.fat_size_16);
43     DBG_PRINT("[ fat12          ] Sectors Per Track: %u\n",
44         ↪ fs->bs.bpb.sectors_per_track);
45     DBG_PRINT("[ fat12          ] Number of Heads: %u\n",
46         ↪ fs->bs.bpb.number_of_heads);
47     DBG_PRINT("[ fat12          ] Hidden Sectors: %u\n",
48         ↪ fs->bs.bpb.hidden_sectors);
49     DBG_PRINT("[ fat12          ] Total Sectors (32): %u\n",
50         ↪ fs->bs.bpb.total_sectors_32);

```

```

41     DBG_PRINT("[ fat12          ] Drive Number: 0x%02x\n",
    ↪ fs->bs.extended_bpb.drive_number);
42     DBG_PRINT("[ fat12          ] Boot Signature: 0x%02x\n",
    ↪ fs->bs.extended_bpb.boot_signature);
43     DBG_PRINT("[ fat12          ] Volume ID: 0x%08x\n",
    ↪ fs->bs.extended_bpb.volume_id);
44     DBG_PRINT("[ fat12          ] Volume Label: %.11s\n",
    ↪ fs->bs.extended_bpb.volume_label);
45     DBG_PRINT("[ fat12          ] File System Type: %.8s\n",
    ↪ fs->bs.extended_bpb.file_system_type);
46     DBG_PRINT("[ fat12          ] Signature: 0x%04x\n", fs->bs.signature);
47
48     /* --- NEW --- */
49     fs->fat_lba = fs->bs.bpb.reserved_sector_count;
50     fs->fat_sectors = fs->bs.bpb.num_fats * fs->bs.bpb.fat_size_16;
51     fs->root_dir_lba = fs->fat_lba + fs->fat_sectors;
52     fs->root_dir_sectors = ((fs->bs.bpb.root_entry_count *
    ↪ DIRECTORY_ENTRY_SIZE) + (fs->bs.bpb.bytes_per_sector - 1)) /
    ↪ fs->bs.bpb.bytes_per_sector;
53
54     DBG_PRINT("[ fat12          ] FAT start LBA: %u\n", fs->fat_lba);
55     DBG_PRINT("[ fat12          ] FAT size (sectors): %u\n", fs->fat_sectors);
56     DBG_PRINT("[ fat12          ] Root dir start LBA: %u\n", fs->root_dir_lba);
57     DBG_PRINT("[ fat12          ] Root dir size (sectors): %u\n",
    ↪ fs->root_dir_sectors);
58     /* --- END NEW --- */
59
60     return fs;
61 }

```

The formulas match the ones above – computed once, cached forever. We also add them to our debug output, so we can inspect them and use them as the starting point for manual hexdumps. `fat_lba` and `fat_sectors` will be used in Chapter 5 when we decode the FAT itself; for now they are the building blocks for `root_dir_lba`.

Peeking Inside the Root Directory

Now that we know where the root directory lives and that it holds a sequence of 32-byte directory entries, let's take a look inside.

But first, let's create some actual files and directories inside the root directory so we have something to look at. For that we use a loopback mount – it treats our disk image as if it were a real block device, letting Linux's own FAT driver take over and giving us ordinary tools to create files and directories

on it, exactly as it would on a physical disk. We will be able to write files with our own code soon enough; for now, this is the only way we have to put files on our disk image, and it doubles as a check that our implementation reads them back correctly.

```

1  # Mount disk.img as a loopback device at /mnt/fat12
2  sudo mkdir -p /mnt/fat12
3  sudo mount -o loop disk.img /mnt/fat12
4
5  # Create a couple of files and a directory on it
6  echo "Hello from FAT12!" | sudo tee /mnt/fat12/HELLO.TXT
7  echo "Some text file" | sudo tee /mnt/fat12/README.TXT
8  sudo mkdir /mnt/fat12/MYDIR
9
10 # Unmount so the writes are flushed back to disk.img
11 sudo umount /mnt/fat12

```

Now that there are files and directories inside the root directory, let's take a look at its raw contents with `xxd`. For that we need the byte offset where the root directory starts inside `disk.img`. Everything we need to calculate it is already sitting in our debug output – let's build and run once to see it:

```

1  DEBUG=1 ./build.sh
2  ./fat12-cli

```

We should see something like:

```

1  [ fat12          ] --- fat12_mount() ---
2  [ block_device ] read 1 sector(s) starting at LBA 0
3  [ file_io       ] read 512 bytes at 0x0
4  [ fat12         ] OEM Name: mkfs.fat
5  [ fat12         ] Bytes Per Sector: 512
6  [ fat12         ] Sectors Per Cluster: 2
7  [ fat12         ] Reserved Sector Count: 1
8  [ fat12         ] Number of FATs: 2
9  [ fat12         ] Root Entry Count: 512
10 [ fat12         ] Total Sectors (16): 8192
11 [ fat12         ] Media Descriptor: 0xf8
12 [ fat12         ] FAT Size (sectors): 12
13 [ fat12         ] Sectors Per Track: 32
14 [ fat12         ] Number of Heads: 2
15 [ fat12         ] Hidden Sectors: 0
16 [ fat12         ] Total Sectors (32): 0
17 [ fat12         ] Drive Number: 0x80

```

```

18 [ fat12          ] Boot Signature: 0x29
19 [ fat12          ] Volume ID: 0x72bae6ba
20 [ fat12          ] Volume Label: MYDISK
21 [ fat12          ] File System Type: FAT12
22 [ fat12          ] Signature: 0xaa55
23 [ fat12          ] FAT start LBA: 1
24 [ fat12          ] FAT size (sectors): 24
25 [ fat12          ] Root dir start LBA: 25
26 [ fat12          ] Root dir size (sectors): 32

```

The last four lines are new – everything above them is the same boot sector output from Chapter 2. Plug the `Root dir start LBA` and `Bytes Per Sector` values from that output into the formula below to get the byte offset:

```
1 Offset = Root dir start LBA * Bytes Per Sector
```

Replace `<offset>` with your calculated value to dump 128 bytes (four entries) from the start of the root directory:

```
1 xxd -g 1 -s <offset> -l 128 disk.img
```

```

1 00003200: 4d 59 44 49 53 4b 20 20 20 20 20 08 00 00 51 7b MYDISK      ...Q{
2 00003210: 3c 5d 3c 5d 00 00 51 7b 3c 5d 00 00 00 00 00 00 <]<]...Q{<].....
3 00003220: 48 45 4c 4c 4f 20 20 20 54 58 54 20 00 98 61 7d HELLO    TXT  ..a}
4 00003230: 3d 5d 3d 5d 00 00 61 7d 3d 5d 03 00 12 00 00 00 =]=]...a}=].....
5 00003240: 52 45 41 44 4d 45 20 20 54 58 54 20 00 67 64 7d README  TXT  .gd}
6 00003250: 3d 5d 3d 5d 00 00 64 7d 3d 5d 04 00 0f 00 00 00 =]=]...d}=].....
7 00003260: 4d 59 44 49 52 20 20 20 20 20 20 10 00 91 66 7d MYDIR      ...f}
8 00003270: 3d 5d 3d 5d 00 00 66 7d 3d 5d 05 00 00 00 00 00 =]=]...f}=].....

```

Every line of `xxd` output is exactly 16 bytes, and each directory entry is 32 bytes – so two lines together make up exactly one entry. We can already recognize the names of the files and directories we created. But to really understand what we are looking at, we first need to know how a directory entry is structured.

Anatomy of a Directory Entry

Each of those 32-byte entries is structured like the diagram below:

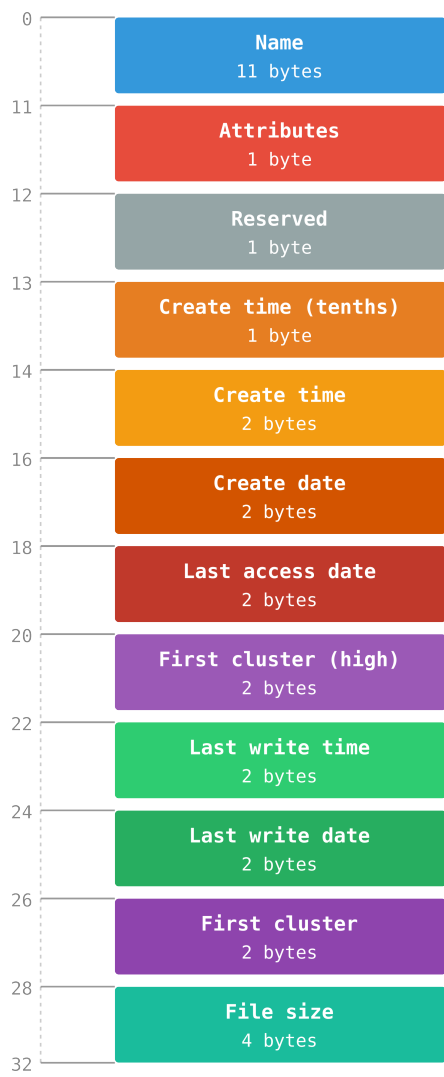


Figure 7. Directory entry layout

Let's first mirror this layout into a C struct, before we go into detail on each field. Like every on-disk struct, its byte layout has to match exactly, so it goes between the `#pragma pack(push, 1)` and `#pragma pack(pop)` pair:

src/layout.h

```
1  typedef struct {
2      char name[11];
3      uint8_t attributes;
4      uint8_t reserved;
5      uint8_t create_time_tenth;
6      uint16_t create_time;
7      uint16_t create_date;
8      uint16_t last_access_date;
9      uint16_t first_cluster_high;
10     uint16_t last_write_time;
11     uint16_t last_write_date;
12     uint16_t first_cluster;
13     uint32_t file_size;
14 } DirectoryEntry;
```

Let's take a look at every field. `name`, `attributes`, and the timestamp fields each get their own section later in this chapter, so we skip them here. `reserved` is set aside for Windows NT and never touched by us. `file_size` is exactly what it sounds like: the file's size in bytes, straight from the entry. `first_cluster` is the one field we are not covering yet – it gets its own explanation in Chapter 4. `first_cluster_high` is only used on FAT32, so we can ignore it entirely.

The 8.3 Name Format

The name is stored in one 11-byte wide field. But it isn't a plain name: it uses a special encoding called 8.3, where the first 8 bytes hold the actual name and the last 3 bytes hold an optional extension. Unused space in either part is padded with spaces (0x20), and the dot between name and extension is never stored on disk – it's only implied, and only inserted back on read if the extension isn't empty. The name is stored in ASCII, so each byte holds exactly one character – which is where the “8.3” name comes from: 8 characters for the name, 3 for the extension.

Here are just the 11 name bytes from each of the three entries from the hexdump above:

```

1  48 45 4c 4c 4f 20 20 20 54 58 54  HELLO  TXT
2  52 45 41 44 4d 45 20 20 54 58 54  README TXT
3  4d 59 44 49 52 20 20 20 20 20 20  MYDIR

```

No dot in sight – 54 58 54 (TXT) sits straight after the padding, and MYDIR trails off into six 0x20 bytes with no extension at all.

Only uppercase letters, digits, and a handful of punctuation marks are legal – no lowercase, and no spaces except as padding. The spec also allows the full extended byte range 0x80-0xFF, using whatever OEM code page was active when the entry was created.

The first byte of the name also has a special meaning of its own, which we will see later in this chapter. For now, it's enough to know that the value 0xE5 is one such special byte – because it causes a gotcha for the name itself: 0xE5 is also a valid lead byte for Kanji characters in the Shift-JIS code page used in Japan, so a perfectly good filename could start with that exact byte. The FAT spec reroutes it: a real first byte of 0xE5 gets stored as 0x05 instead, the ASCII ENQ control character, never valid in a filename, so there's no ambiguity.

Now that we understand how a name is encoded, we can build a function that decodes it – turning the padded 8.3 format back into the plain name.ext string.

src/fat12.c

```

1  static void decode_8_3_name(const DirectoryEntry* entry, char* out)
2  {
3      int i = 0;
4
5      /* 0x05 escape – real first byte is 0xE5 */
6      if (entry->name[0] == 0x05)
7          out[i++] = (char)0xE5;
8
9      /* Copy name bytes until padding or end */
10     for (; i < 8 && entry->name[i] != ' '; i++)
11         out[i] = entry->name[i];
12
13     /* If extension exists, insert dot and copy non-space bytes */
14     const char* extension = entry->name + 8;
15     int has_extension = extension[0] != ' ';
16     if (has_extension)
17     {
18         out[i] = '.';

```

```

19     i++;
20     for (int k = 0; k < 3; k++)
21     {
22         if (extension[k] != ' ')
23         {
24             out[i] = extension[k];
25             i++;
26         }
27     }
28 }
29
30 out[i] = '\0';
31 }

```

The function walks the raw 11 bytes once, left to right: it un-escapes a leading 0x05 back into 0xE5, copies name bytes until it hits padding, and – only if the extension isn’t blank – appends a dot and its non-space characters. A final \0 terminates the string.

The Attribute Byte

The attribute byte is a bitfield describing properties of the entry. Each bit and what it means when set (1) is listed in the table below:

Bit	Flag	Meaning
0	Read-only	Protects against writes and modification. Most real-world implementations also block deletion, even though that isn’t officially specified.
1	Hidden	Hides the entry from normal directory listings.
2	System	An indicator that this is an OS-critical file – the spec defines no enforcement at all. Any protection is a convention individual tools layer on top, not something FAT12 requires.
3	Volume Label	Marks the entry as the volume label (more on this later).
4	Directory	Marks the entry as a directory.

Bit	Flag	Meaning
5	Archive	Marks the file as changed since the last backup. Set whenever a file is created, renamed, or written to; cleared by backup software once it has backed the file up – a cheap alternative to comparing timestamps or checksums to find what changed.

Bits 6 and 7 are reserved – their value should be ignored.

Our directory iteration always returns entries with Hidden or System set – we never filter on either flag. That is not a shortcut: hiding them is a cosmetic choice for a listing tool, not something the filesystem enforces. Even `ls -a` on Linux shows them.

Some combinations of flags are worth calling out on their own, since their meaning is not just the sum of the individual bits:

Flags	Meaning
Directory and Volume Label both clear (0)	Marks the entry as a file.
Read-only, Hidden, System, and Volume Label all set (1)	Marks the entry as an LFN fragment – not a valid attribute combination for a real entry, it’s a sentinel for the Long Filename (LFN / VFAT) extension. We filter these out and ignore them throughout the book; Appendix B: Long Filename Support (LFN / VFAT) explains how they actually work.

Let’s define constants for each flag:

src/fat12.c

```
1 #define FAT12_ATTR_READ_ONLY    0x01
2 #define FAT12_ATTR_HIDDEN      0x02
3 #define FAT12_ATTR_SYSTEM      0x04
4 #define FAT12_ATTR_VOLUME_ID   0x08
5 #define FAT12_ATTR_DIRECTORY   0x10
6 #define FAT12_ATTR_ARCHIVE     0x20
```



FAT12_ATTR_VOLUME_ID, not FAT12_ATTR_VOLUME_LABEL? The FAT spec itself names this constant ATTR_VOLUME_ID, even though the entry it marks is the volume label.

The Timestamp Fields

Each directory entry carries six time-related fields in total. Two of them we leave unused, like most implementations: `create_time_tenth`, a byte holding 0-199 centiseconds of sub-second creation precision, and `last_access_date`, which records the date a file was last read. Sub-second creation precision is rarely useful to any application, and updating `last_access_date` on every read costs an extra disk write for a value few tools ever check.

The other four make up two timestamp pairs we actually need: creation time (`create_time` / `create_date`) and last modification time (`last_write_time` / `last_write_date`). Each pair shares the same two bit-packed 2-byte fields.

The time field packs hours, minutes, and seconds into 16 bits:

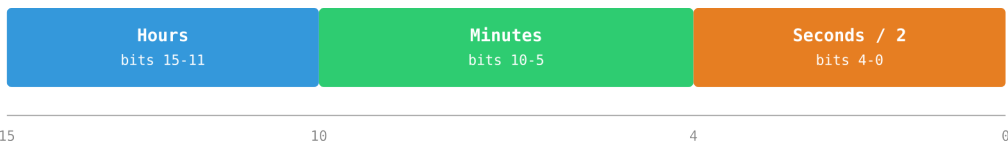


Figure 8. Time field bit layout

Seconds only get 5 bits – not enough to count 0-59 – so FAT stores them in 2-second increments instead; multiply the stored value by 2 to recover real seconds. A FAT timestamp can never land on an odd second.

The date field packs year, month, and day the same way:

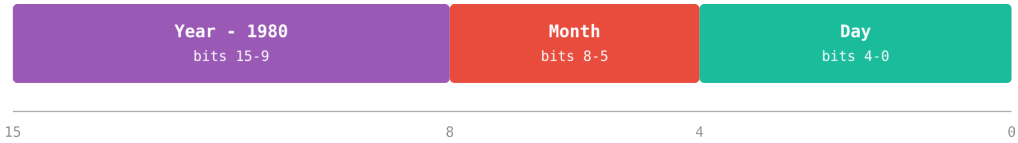


Figure 9. Date field bit layout

Instead of a full year, it counts years since the MS-DOS epoch of January 1, 1980; add that back to get the real year.

A time field and a date field together give us a full timestamp, which we want to hand callers as a single struct.

include/fat12.h

```
1 typedef struct {
2     unsigned year;
3     unsigned month;
4     unsigned day;
5     unsigned hours;
6     unsigned minutes;
7     unsigned seconds;
8 } Timestamp;
```

Now we can write the function that decodes both into a Timestamp struct:

src/fat12.c

```
1 static Timestamp decode_timestamp(uint16_t time, uint16_t date)
2 {
3     Timestamp ts;
4
5     ts.hours   = (time >> 11) & 0x1F;
6     ts.minutes = (time >> 5)  & 0x3F;
7     ts.seconds = (time & 0x1F) * 2;
8
9     ts.day     = date & 0x1F;
10    ts.month    = (date >> 5) & 0x0F;
11    ts.year     = ((date >> 9) & 0x7F) + 1980;
12
13    return ts;
14 }
```

The shifts and masks simply pull out each component we saw in the field maps above; the only extra work is the $\times 2$ and $+ 1980$ correcting seconds and year back to their real values.

Reading the Root Directory

We know where the root directory lives, how big it is, and what its bytes mean. None of that helps until it is actually sitting in memory – let’s read it:

src/fat12.c

```
1 static DirectoryEntry* read_root_directory(FAT12FS* fs)
2 {
3     DBG_PRINT("[ fat12          ] read root directory\n");
4     uint32_t bytes = fs->root_dir_sectors * fs->bs.bpb.bytes_per_sector;
5     DirectoryEntry* entries = (DirectoryEntry*)malloc(bytes);
6     block_device_read(fs->device, fs->root_dir_lba, fs->root_dir_sectors,
7         ↪ entries);
8     return entries;
9 }
```

`read_root_directory` allocates a buffer sized to the root directory region and reads it in one call using the block device. Because `DirectoryEntry` is packed to match the on-disk layout, the bytes read can be used directly as an array of entries.

Filtering the Entries

We now have the contents of the root directory as an array of directory entries, but that array still contains entries that should never be visible to a caller – free slots, LFN fragments, and invalid entries.

Two special values in an entry’s first name byte drive most of this filtering. `0x00` marks the end of the directory: nothing beyond it is ever allocated, so a scan can stop right there. `0xE5` also marks an entry as free, but only that one entry – real entries can still follow it, so a scan has to skip it and keep going. It’s the same byte that forced the `0x05` escape we saw earlier.



Deleting a file (Chapter 9) only clears that one entry's first byte to 0xE5, leaving a hole behind – the entries after it are still real and in use. That hole is free but not the end of the directory, which is exactly why the two need separate markers.

To filter those entries out, let's introduce a few small helper functions, plus the function that ties them together and returns the next valid entry from an array of directory entries:

src/fat12.c

```

1  #include <stdbool.h>
2
3  #define NAME_FREE  0xE5
4  #define NAME_END   0x00
5
6  static bool is_free_entry(const DirectoryEntry* entry)
7  {
8      // Cast to unsigned char: 0xE5 doesn't fit a signed char (-128 to 127),
9      // so without the cast, sign-extension turns it into 0xFFFFFE5 and the
10     // comparison fails.
11     return (unsigned char)entry->name[0] == NAME_FREE;
12 }
13
14 static bool is_end_of_directory(const DirectoryEntry* entry)
15 {
16     return entry->name[0] == NAME_END;
17 }
18
19 static bool is_lfn_fragment(const DirectoryEntry* entry)
20 {
21     /* Bits 6 and 7 of the attribute byte are reserved, so mask them
22      off (0x3F) before comparing instead of testing for an exact match. */
23     return (entry->attributes & 0x3F) == (FAT12_ATTR_READ_ONLY |
24     ↪ FAT12_ATTR_HIDDEN | FAT12_ATTR_SYSTEM | FAT12_ATTR_VOLUME_ID);
25 }
26
27 static bool is_invalid_entry(const DirectoryEntry* entry)
28 {
29     /* Directory and Volume Label together mean neither – the FAT spec's
30      own validation algorithm treats this combination as corruption,
31      never something a well-formed volume produces. */
32     uint8_t mask = FAT12_ATTR_DIRECTORY | FAT12_ATTR_VOLUME_ID;
33     return (entry->attributes & mask) == mask;
34 }

```

```
35 static DirectoryEntry* next_valid_entry(  
36     DirectoryEntry* entries,  
37     uint32_t count,  
38     uint32_t* offset  
39 )  
40 {  
41     while (*offset < count)  
42     {  
43         DirectoryEntry* entry = &entries[*offset];  
44         if (is_end_of_directory(entry)) return NULL;  
45  
46         (*offset)++;  
47  
48         if (is_free_entry(entry)) continue;  
49         if (is_lfn_fragment(entry)) continue;  
50         if (is_invalid_entry(entry)) continue;  
51  
52         return entry;  
53     }  
54  
55     return NULL;  
56 }
```

`next_valid_entry` walks the `entries` array starting at `*offset`, skipping free slots, LFN fragments, and invalid entries along the way, and stops the moment it hits the end-of-directory marker. The first real entry it finds is returned directly, with `*offset` left pointing just past it – so calling it again picks up right where the last call left off. It returns `NULL` when there is nothing left to give.

Putting It All Together

We have every building block. Now it is time to put it all together: a directory iteration API – open a directory by path (root only for now, Chapter 6 extends this to walk into subdirectories), read the next entry if one is present, and close it when done. Let's start by extending the File System API:

`include/fat12.h`

```
1  typedef enum {
2      ENTRY_FILE,
3      ENTRY_DIRECTORY,
4      ENTRY_VOLUME_LABEL
5  } EntryKind;
6
7  typedef struct {
8      char name[13];
9      uint32_t size;
10     EntryKind kind;
11     Timestamp create_time;
12     Timestamp modify_time;
13 } DirEntry;
14
15 typedef struct Directory Directory;
16
17 Directory* fat12_opendir(FAT12FS* fs, const char* path);
18 int fat12_readdir(Directory* dir, DirEntry* out);
19 void fat12_closedir(Directory* dir);
```

`DirEntry` is the directory entry we actually expose to callers – a smaller, already-decoded view of what a raw directory entry holds, with only the fields anyone outside the FAT12 implementation should ever need. `Directory` is the opaque handle `fat12_opendir` hands back: only declared to exist, with everything it actually contains left to the implementation, the same pattern C’s own `FILE*` uses. Callers hand that handle to `fat12_readdir`, which writes the next entry into `out`, and to `fat12_closedir`, which closes it and frees whatever resources it was holding – without either of them ever needing to know what is inside. The point is to hide details the caller has no business knowing – how we track iteration state is our concern, not theirs.

Now that the public API is declared, let’s define how we actually track that state inside `Directory`, and implement `fat12_opendir`:

src/fat12.c

```

1  struct Directory {
2      DirectoryEntry* entries;
3      uint32_t count;
4      uint32_t offset;
5  };
6
7  Directory* fat12_opendir(FAT12FS* fs, const char* path)
8  {
9      DBG_PRINT("[ fat12          ] --- fat12_opendir(\"%s\") ---\n", path);
10
11     /* Guards against anything but the root directory for now */
12     if (strcmp(path, "/") != 0) return NULL;
13
14     Directory* dir = (Directory*)malloc(sizeof(Directory));
15     dir->entries = read_root_directory(fs);
16     dir->count = fs->bs.bpb.root_entry_count;
17     dir->offset = 0;
18
19     return dir;
20 }

```

`fat12_opendir` reads the full root directory into memory, allocates the `Directory` handle, and initializes it with that array of entries, the total count and an offset starting at 0 for the scan. It currently guards against every path except the root directory – every other path is rejected with `NULL`, since we have no way yet to resolve anything deeper. Chapter 6 replaces this check with a real path resolver that walks into subdirectories.

Let's continue with `fat12_readdir`:

src/fat12.c

```

1  int fat12_readdir(Directory* dir, DirEntry* out)
2  {
3      DBG_PRINT("[ fat12          ] --- fat12_readdir() ---\n");
4      DirectoryEntry* entry = next_valid_entry(dir->entries, dir->count,
5      ↪ &dir->offset);
6      if (entry == NULL) return -1;
7
8      decode_8_3_name(entry, out->name);
9      out->size = entry->file_size;
10     out->create_time = decode_timestamp(entry->create_time,
11     ↪ entry->create_date);
12     out->modify_time = decode_timestamp(entry->last_write_time,
13     ↪ entry->last_write_date);
14 }

```

```
12     if (entry->attributes & FAT12_ATTR_VOLUME_ID)
13         out->kind = ENTRY_VOLUME_LABEL;
14     else if (entry->attributes & FAT12_ATTR_DIRECTORY)
15         out->kind = ENTRY_DIRECTORY;
16     else
17         out->kind = ENTRY_FILE;
18
19     return 0;
20 }
```

`fat12_readdir` looks at where it currently is in the scan and checks whether another entry follows. If not, it returns `-1` – end of directory. Otherwise, it decodes the raw entry it found into the caller-facing struct, setting every other field we expose – including `kind`, translated once here from the raw attribute byte.

The last thing missing is `fat12_closedir`:

src/fat12.c

```
1 void fat12_closedir(Directory* dir)
2 {
3     free(dir->entries);
4     free(dir);
5 }
```

`fat12_closedir` simply frees the memory occupied by the directory entries and the directory handle itself.

The `ls` Command

Now that we have the directory iteration API, let's replace `main` from Chapter 2 with a command-line interface – the boilerplate future chapters will extend, rather than replace, to add more CLI commands:

platform/cli/main.c

```

1  #include <stdio.h>
2  #include <stdint.h>
3  #include <string.h>
4  #include <stdlib.h>
5  #include "block_device.h"
6  #include "fat12.h"
7  #include "file_block_device.h"
8
9  static int cmd_ls(FAT12FS *fs, int argc, char *argv[])
10 {
11     if (argc != 3)
12     {
13         fprintf(stderr, "usage: ls <path>\n");
14         return 1;
15     }
16
17     const char *path = argv[2];
18     Directory *dir = fat12_opendir(fs, path);
19     if (dir == NULL)
20     {
21         fprintf(stderr, "error: directory not found\n");
22         return 1;
23     }
24
25     printf("%-12s  %-9s  %-19s  %-19s\n", "NAME", "TYPE/SIZE", "CREATED",
26         ↪ "MODIFIED");
27
28     DirEntry entry;
29     while (fat12_readdir(dir, &entry) == 0)
30     {
31         char type_str[16];
32
33         if (entry.kind == ENTRY_VOLUME_LABEL)
34             snprintf(type_str, sizeof(type_str), "<VOL>");
35         else if (entry.kind == ENTRY_DIRECTORY)
36             snprintf(type_str, sizeof(type_str), "<DIR>");
37         else
38             snprintf(type_str, sizeof(type_str), "%u B", entry.size);
39
40         printf(
41             "%-12s  %-9s  %04u-%02u-%02u %02u:%02u:%02u  %04u-%02u-%02u\n",
42             ↪ %02u:%02u:%02u\n",
43             entry.name, type_str,
44             entry.create_time.year, entry.create_time.month,
45             ↪ entry.create_time.day,
46             entry.create_time.hours, entry.create_time.minutes,
47             ↪ entry.create_time.seconds,
48             entry.modify_time.year, entry.modify_time.month,
49             ↪ entry.modify_time.day,

```

```

45         entry.modify_time.hours, entry.modify_time.minutes,
46         ↪ entry.modify_time.seconds
47     );
48 }
49 fat12_closedir(dir);
50 return 0;
51 }
52
53 int main(int argc, char *argv[])
54 {
55     if (argc < 2)
56     {
57         fprintf(stderr, "usage: <command>\n");
58         return 1;
59     }
60
61     /* Hardcoded so every command stays short */
62     BlockDevice *device = file_block_device_open("disk.img");
63     if (device == NULL)
64     {
65         fprintf(stderr, "error: could not open disk image\n");
66         return 1;
67     }
68
69     FAT12FS *fs = fat12_mount(device);
70
71     char *command = argv[1];
72     int ret = 0;
73
74     if (strcmp(command, "ls") == 0) {
75         ret = cmd_ls(fs, argc, argv);
76     } else {
77         fprintf(stderr, "unknown command: %s\n", command);
78         ret = 1;
79     }
80
81     fat12_umount(fs);
82     block_device_close(device);
83
84     return ret;
85 }

```

main opens the disk image and mounts the filesystem before dispatching to a command, then unmounts and closes it again once that command returns. The interesting part is `cmd_ls`: typing `ls` followed by a path opens that directory – only `/` works for now, since path walking arrives in Chapter 6 – then walks every entry with `fat12_readdir`, printing one line per entry – its

name, type or size, and both timestamps.

Our disk image still contains the entries we created manually earlier in this chapter. Let's build and run our brand new `ls` command using `/` to list the root directory:

```
1 ./build.sh
2 ./fat12-cli ls /
```

```
1 NAME          TYPE/SIZE  CREATED          MODIFIED
2 MYDISK        <VOL>     2026-09-28 15:26:34 2026-09-28 15:26:34
3 HELLO.TXT     18 B      2026-09-29 15:43:02 2026-09-29 15:43:02
4 README.TXT    15 B      2026-09-29 15:43:08 2026-09-29 15:43:08
5 MYDIR         <DIR>     2026-09-29 15:43:12 2026-09-29 15:43:12
```

It worked! Both `HELLO.TXT` and `README.TXT` show up with the correct file size, and their creation and modification times (yours will differ).

Our directory, `MYDIR`, shows up too – correctly marked `<DIR>` instead of a size.

The only entry we didn't create ourselves is `MYDISK` – it's the very first line, marked `<VOL>`.



Why is the volume label stored as a directory entry? Because it's the original mechanism – FAT reuses the 8.3 name field already built into every directory entry, marked with the `Volume Label` bit. The Extended BPB's own copy (the `Volume Label` field from Chapter 2's debug output) came later, and the spec expects drivers to keep it in sync with the directory entry, not the other way around. It's also typically the very first entry in the root directory, since formatting writes it there before any files exist.

What's Next

We can now list the root directory's contents. But it only tells us *what* files are there, not what they actually contain. That's where `first_cluster` comes in – a field we have read into every `DirectoryEntry` this whole chapter but never followed. It is the connection between a directory entry and its actual data. Chapter 4 shows how we follow it to read a file's contents.

Chapter 4: Reading Files

Chapter 3 showed us which files are on the volume. But a listing only gives us names and sizes, not a single byte of what those files actually hold. The link to their contents is a field we have read into every directory entry but never followed: `first_cluster`. By the end of this chapter, we will have followed it all the way to a file's contents, with our own `cat` command reading a file straight off the volume and printing it, byte-for-byte.

The Data Region

`first_cluster` points into the **data region**, which we already saw in Chapter 3's volume layout diagram:

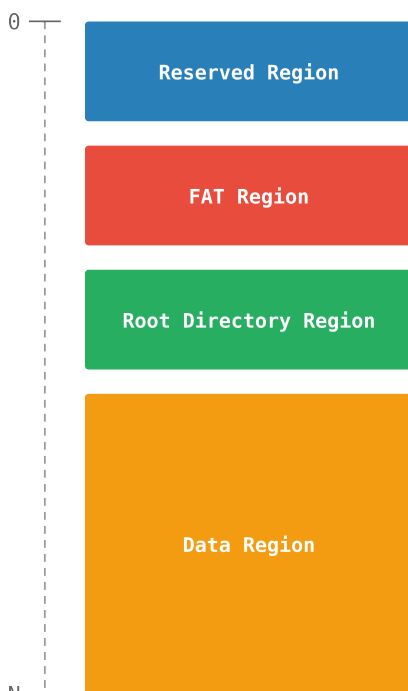


Figure 10. FAT12 volume layout – full

The data region is the last region of the volume, running all the way to its final sector. To follow `first_cluster`, we need to know where the region starts. As the diagram shows, the region right before it is the root directory region, and we already calculated both its start and its size in Chapter 3. Adding the two gives us the data region's start:

```
1 Data region start LBA = Root dir start LBA + Root directory size (sectors)
```

We cache it in `FAT12FS` and compute it once on mount, right after the root directory values it builds on:

src/fat12.c

```
1  struct FAT12FS {
2      BlockDevice* device;
3      BootSector bs;
4      uint32_t fat_lba;
5      uint32_t fat_sectors;
6      uint32_t root_dir_lba;
7      uint32_t root_dir_sectors;
8
9      /* --- NEW --- */
10     uint32_t data_lba;
11 };
12
13 // ...
14
15 FAT12FS* fat12_mount(BlockDevice* device)
16 {
17     DBG_PRINT("[ fat12          ] --- fat12_mount() ---\n");
18     FAT12FS* fs = malloc(sizeof(FAT12FS));
19     fs->device = device;
20     fs->bs = read_boot_sector(device);
21
22     /* jump, boot_code, reserved_nt: not used by FAT itself, so we skip them */
23     DBG_PRINT("[ fat12          ] OEM Name: %.8s\n", fs->bs.oem_name);
24     DBG_PRINT("[ fat12          ] Bytes Per Sector: %u\n",
25         ↪ fs->bs.bpb.bytes_per_sector);
26     DBG_PRINT("[ fat12          ] Sectors Per Cluster: %u\n",
27         ↪ fs->bs.bpb.sectors_per_cluster);
28     DBG_PRINT("[ fat12          ] Reserved Sector Count: %u\n",
29         ↪ fs->bs.bpb.reserved_sector_count);
30     DBG_PRINT("[ fat12          ] Number of FATs: %u\n", fs->bs.bpb.num_fats);
31     DBG_PRINT("[ fat12          ] Root Entry Count: %u\n",
32         ↪ fs->bs.bpb.root_entry_count);
```

```

29     DBG_PRINT("[ fat12          ] Total Sectors (16): %u\n",
    ↪ fs->bs.bpb.total_sectors_16);
30     DBG_PRINT("[ fat12          ] Media Descriptor: 0x%02x\n", fs->bs.bpb.media);
31     DBG_PRINT("[ fat12          ] FAT Size (sectors): %u\n",
    ↪ fs->bs.bpb.fat_size_16);
32     DBG_PRINT("[ fat12          ] Sectors Per Track: %u\n",
    ↪ fs->bs.bpb.sectors_per_track);
33     DBG_PRINT("[ fat12          ] Number of Heads: %u\n",
    ↪ fs->bs.bpb.number_of_heads);
34     DBG_PRINT("[ fat12          ] Hidden Sectors: %u\n",
    ↪ fs->bs.bpb.hidden_sectors);
35     DBG_PRINT("[ fat12          ] Total Sectors (32): %u\n",
    ↪ fs->bs.bpb.total_sectors_32);
36     DBG_PRINT("[ fat12          ] Drive Number: 0x%02x\n",
    ↪ fs->bs.extended_bpb.drive_number);
37     DBG_PRINT("[ fat12          ] Boot Signature: 0x%02x\n",
    ↪ fs->bs.extended_bpb.boot_signature);
38     DBG_PRINT("[ fat12          ] Volume ID: 0x%08x\n",
    ↪ fs->bs.extended_bpb.volume_id);
39     DBG_PRINT("[ fat12          ] Volume Label: %.11s\n",
    ↪ fs->bs.extended_bpb.volume_label);
40     DBG_PRINT("[ fat12          ] File System Type: %.8s\n",
    ↪ fs->bs.extended_bpb.file_system_type);
41     DBG_PRINT("[ fat12          ] Signature: 0x%04x\n", fs->bs.signature);
42
43     fs->fat_lba = fs->bs.bpb.reserved_sector_count;
44     fs->fat_sectors = fs->bs.bpb.num_fats * fs->bs.bpb.fat_size_16;
45     fs->root_dir_lba = fs->fat_lba + fs->fat_sectors;
46     fs->root_dir_sectors = ((fs->bs.bpb.root_entry_count *
    ↪ DIRECTORY_ENTRY_SIZE) + (fs->bs.bpb.bytes_per_sector - 1)) /
    ↪ fs->bs.bpb.bytes_per_sector;
47
48     /* --- NEW --- */
49     fs->data_lba = fs->root_dir_lba + fs->root_dir_sectors;
50
51     DBG_PRINT("[ fat12          ] FAT start LBA: %u\n", fs->fat_lba);
52     DBG_PRINT("[ fat12          ] FAT size (sectors): %u\n", fs->fat_sectors);
53     DBG_PRINT("[ fat12          ] Root dir start LBA: %u\n", fs->root_dir_lba);
54     DBG_PRINT("[ fat12          ] Root dir size (sectors): %u\n",
    ↪ fs->root_dir_sectors);
55
56     /* --- NEW --- */
57     DBG_PRINT("[ fat12          ] Data region start LBA: %u\n", fs->data_lba);
58
59     return fs;
60 }

```

Build and run with debug output enabled:

```
1 DEBUG=1 ./build.sh
2 ./fat12-cli ls /
```

Among the output is the new line we just added, telling us at which LBA the data region starts on our disk image:

```
1 [ fat12          ] Data region start LBA: 57
```

Carving Up the Data Region

The data region is where the contents of every file on the volume actually live. It is nothing more than one long array of numbered, equally-sized chunks called **clusters**. Each cluster is a contiguous span of one or more sectors, the smallest amount of space a file can occupy. Our disk image uses 2 sectors per cluster (set by the `-s 2` flag in `create_disk.sh`'s `mkfs.fat` command back in Chapter 1). Laid out end to end, our data region looks like this:

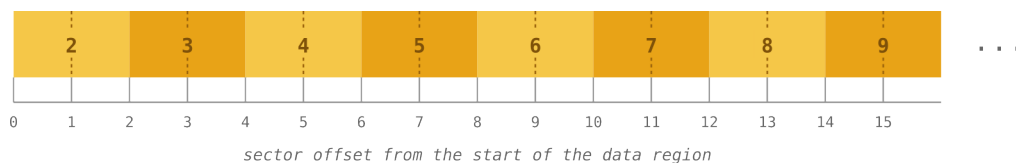


Figure 11. Data region as numbered clusters, 2 sectors each

Each box is one cluster, labeled with its number, and the scale underneath counts sectors from the start of the data region. The clusters are numbered sequentially, each spanning exactly 2 sectors: cluster 2 covers sectors 0–1, cluster 3 covers sectors 2–3, and so on. Notice that the numbering starts at 2, not 0. That is no typo. Cluster numbers 0 and 1 are reserved and have no space of their own in the data region (we will see why in Chapter 5), so the first real cluster, cluster 2, sits right at the data region's very first sector.

A file's contents are cut into cluster-sized pieces, one piece per cluster, and each cluster holds nothing but its raw piece of data. A small file fits in a single cluster, a bigger one takes as many as it needs (finding the rest of them is Chapter 5's job). So every file on the volume consumes a whole number of clusters, even a tiny one. Whenever a file's size is not an exact multiple of the

cluster size, the leftover bytes are still allocated to the file, even though they hold none of its data. That unused remainder is called **slack space**.

`first_cluster` holds exactly such a cluster number: the number of the first cluster containing the file's data.

Finding a File's First Cluster

Now that we know what `first_cluster` holds, let's follow it by hand. First, we need its value from a file's directory entry, so let's look at the start of the root directory again, the same way we did in Chapter 3. As a reminder, its byte offset comes from the `Root dir start LBA` and `Bytes Per Sector` values in our debug output:

```
1 Byte offset = Root dir start LBA * Bytes per sector
```

Replace `<offset>` with your calculated byte offset:

```
1 xxd -g 1 -s <offset> -l 128 disk.img
```

```
1 00003200: 4d 59 44 49 53 4b 20 20 20 20 20 08 00 00 51 7b MYDISK      ...Q{
2 00003210: 3c 5d 3c 5d 00 00 51 7b 3c 5d 00 00 00 00 00 00 <]<]...Q{<].....
3 00003220: 48 45 4c 4c 4f 20 20 20 54 58 54 20 00 98 61 7d HELLO  TXT  ..a}
4 00003230: 3d 5d 3d 5d 00 00 61 7d 3d 5d 03 00 12 00 00 00 =]=]..a}=].....
5 00003240: 52 45 41 44 4d 45 20 20 54 58 54 20 00 67 64 7d README  TXT  .gd}
6 00003250: 3d 5d 3d 5d 00 00 64 7d 3d 5d 04 00 0f 00 00 00 =]=]..d}=].....
7 00003260: 4d 59 44 49 52 20 20 20 20 20 20 10 00 91 66 7d MYDIR      ...f}
8 00003270: 3d 5d 3d 5d 00 00 66 7d 3d 5d 05 00 00 00 00 00 =]=]..f}=].....
```

Focus on the third and fourth rows, `HELLO.TXT`'s entry. As Chapter 3's directory entry layout showed, `first_cluster` sits at offset 26 of the 32-byte entry and `file_size` at offset 28, together filling the entry's last six bytes. So both sit at the end of its second row: `03 00` is `first_cluster`, and `12 00 00 00` is `file_size`. (Your dump may show different values; if so, use yours from here on.) Like every multi-byte value in FAT12, both are stored little-endian, so we read their bytes back to front:

```

1 first_cluster: 03 00      -> 0x0003      = 3
2 file_size:    12 00 00 00 -> 0x00000012 = 18

```

So HELLO.TXT starts at cluster 3 and is 18 bytes long.

Now let's find where that cluster lives on our volume. We need two numbers for that: the data region's start LBA, which our debug output already printed earlier in this chapter, and the cluster number itself, which is simply `first_cluster` (3 in our example). Cluster 2 sits right at the data region's start, and every cluster after it follows directly behind the one before:

```

1 Cluster LBA = Data region start LBA + (cluster - 2) * Sectors per cluster

```

Why fixed-size clusters? Because every cluster is the same size, “where is cluster N” collapses into the one-line formula above: multiply and add. The cluster number alone is a complete address, with no size to store next to it. Variable-sized allocation regions would also run into a bigger problem: as files come and go, free space breaks up into gaps of every size, and a new file may not fit into any single gap even when there is plenty of free space in total. Fixed-size clusters rule that out, since any free cluster can hold any piece of any file. (A file's clusters can still end up scattered across the volume, as we will see in Chapter 5, but no free cluster is ever unusable.) The price FAT12 pays for all of this is the slack space we mentioned earlier: even a tiny file occupies a whole cluster.

We already know each cluster on our disk image is 2 sectors wide, because we chose that ourselves when we created it. But our code must not assume it, since another volume may use a different cluster size. The value has to come from the volume itself, and it does: the BPB field `Sectors Per Cluster` holds exactly this number, and it has been sitting in our debug output since Chapter 2, waiting for us to need it:

```

1 [ fat12          ] Sectors Per Cluster: 2

```

Now we have every value the formula above needs to calculate the start LBA of any cluster, including HELLO.TXT's first cluster. To dump that cluster with

xxd, we need two more values, both in bytes: its offset (the same way we just did for the root directory) and its size. Bytes Per Sector from our debug output converts both:

```
1 Byte offset          = Cluster LBA * Bytes per sector
2 Cluster size (bytes) = Sectors per cluster * Bytes per sector
```

Now let's dump the whole cluster. Replace <offset> with your byte offset and <cluster_size> with your cluster size in bytes:

```
1 xxd -g 1 -s <offset> -l <cluster_size> disk.img
```

```
1 00007600: 48 65 6c 6c 6f 20 66 72 6f 6d 20 46 41 54 31 32 Hello from FAT12
2 00007610: 21 0a 00 00 00 00 00 00 00 00 00 00 00 00 00 00 !.....
3 00007620: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
4 00007630: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
5 00007640: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
6 00007650: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
7 00007660: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
8 00007670: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
9 00007680: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
10 00007690: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
11 000076a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
12 000076b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
13 000076c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
14 000076d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
15 000076e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
16 000076f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
17 00007700: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
18 00007710: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
19 00007720: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
20 00007730: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
21 00007740: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
22 00007750: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
23 00007760: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
24 00007770: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
25 00007780: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
26 00007790: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
27 000077a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
28 000077b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
29 000077c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
30 000077d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
31 000077e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
32 000077f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
33 00007800: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
```

```

34 00007810: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
35 00007820: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
36 00007830: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
37 00007840: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
38 00007850: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
39 00007860: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
40 00007870: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
41 00007880: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
42 00007890: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
43 000078a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
44 000078b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
45 000078c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
46 000078d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
47 000078e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
48 000078f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
49 00007900: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
50 00007910: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
51 00007920: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
52 00007930: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
53 00007940: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
54 00007950: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
55 00007960: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
56 00007970: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
57 00007980: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
58 00007990: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
59 000079a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
60 000079b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
61 000079c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
62 000079d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
63 000079e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
64 000079f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

As the dump shows, the cluster holds far more bytes than our file needs. Everything after the file's last byte is slack space. Here it happens to be all zeros, but nothing guarantees that: slack can just as well hold garbage, or leftovers from a file that used this cluster before. That is why we always stop reading at `file_size`, never at the end of the cluster. Let's dump the cluster once more, this time asking for only `file_size` bytes. `<offset>` is still the cluster's byte offset; replace `<file_size>` with your file's size from the directory entry:

```
1 xxd -g 1 -s <offset> -l <file_size> disk.img
```

```

1 00007600: 48 65 6c 6c 6f 20 66 72 6f 6d 20 46 41 54 31 32 Hello from FAT12
2 00007610: 21 0a                                     !.

```

`file_size` tells us exactly how many of the cluster's bytes the file really uses, so we can trim the dump to the file's logical end. What is left is the content of `HELLO.TXT`, read straight off our disk image by hand, without a single line of file-reading code.



The last byte, `0a`, is a newline character. `echo` appends one automatically, so when we created `HELLO.TXT` in Chapter 3, its 17 visible characters became an 18-byte file.

Let's capture what we just worked out by hand in code, so it is ready when we put everything together later in this chapter. First up, `data_cluster_to_lba` wraps the Cluster LBA formula in a function that returns the start LBA of any cluster number:

src/fat12.c

```

1 #define FIRST_DATA_CLUSTER 2
2
3 static uint32_t data_cluster_to_lba(FAT12FS* fs, uint16_t cluster)
4 {
5     return fs->data_lba + ((cluster - FIRST_DATA_CLUSTER) *
6     ↪ fs->bs.bpb.sectors_per_cluster);
7 }

```

We also cache the size of a single cluster in bytes on mount. Every cluster we read or write needs a buffer that big, so we will reach for this value again and again throughout the rest of the book. It is simply `Sectors per cluster * Bytes per sector`, the same Cluster size formula we used earlier to tell `xxd` how much to dump:

src/fat12.c

```

1  struct FAT12FS {
2      BlockDevice* device;
3      BootSector bs;
4      uint32_t fat_lba;
5      uint32_t fat_sectors;
6      uint32_t root_dir_lba;
7      uint32_t root_dir_sectors;
8      uint32_t data_lba;
9
10     /* --- NEW --- */
11     uint32_t cluster_bytes;
12 };
13
14 // ...
15
16 FAT12FS* fat12_mount(BlockDevice* device)
17 {
18     DBG_PRINT("[ fat12      ] --- fat12_mount() ---\n");
19     FAT12FS* fs = malloc(sizeof(FAT12FS));
20     fs->device = device;
21     fs->bs = read_boot_sector(device);
22
23     /* jump, boot_code, reserved_nt: not used by FAT itself, so we skip them */
24     DBG_PRINT("[ fat12      ] OEM Name: %.8s\n", fs->bs.oem_name);
25     DBG_PRINT("[ fat12      ] Bytes Per Sector: %u\n",
26         ↪ fs->bs.bpb.bytes_per_sector);
27     DBG_PRINT("[ fat12      ] Sectors Per Cluster: %u\n",
28         ↪ fs->bs.bpb.sectors_per_cluster);
29     DBG_PRINT("[ fat12      ] Reserved Sector Count: %u\n",
30         ↪ fs->bs.bpb.reserved_sector_count);
31     DBG_PRINT("[ fat12      ] Number of FATs: %u\n", fs->bs.bpb.num_fats);
32     DBG_PRINT("[ fat12      ] Root Entry Count: %u\n",
33         ↪ fs->bs.bpb.root_entry_count);
34     DBG_PRINT("[ fat12      ] Total Sectors (16): %u\n",
35         ↪ fs->bs.bpb.total_sectors_16);
36     DBG_PRINT("[ fat12      ] Media Descriptor: 0x%02x\n", fs->bs.bpb.media);
37     DBG_PRINT("[ fat12      ] FAT Size (sectors): %u\n",
38         ↪ fs->bs.bpb.fat_size_16);
39     DBG_PRINT("[ fat12      ] Sectors Per Track: %u\n",
40         ↪ fs->bs.bpb.sectors_per_track);
41     DBG_PRINT("[ fat12      ] Number of Heads: %u\n",
42         ↪ fs->bs.bpb.number_of_heads);
43     DBG_PRINT("[ fat12      ] Hidden Sectors: %u\n",
44         ↪ fs->bs.bpb.hidden_sectors);
45     DBG_PRINT("[ fat12      ] Total Sectors (32): %u\n",
46         ↪ fs->bs.bpb.total_sectors_32);
47     DBG_PRINT("[ fat12      ] Drive Number: 0x%02x\n",
48         ↪ fs->bs.extended_bpb.drive_number);
49     DBG_PRINT("[ fat12      ] Boot Signature: 0x%02x\n",
50         ↪ fs->bs.extended_bpb.boot_signature);

```

```

39     DBG_PRINT("[ fat12          ] Volume ID: 0x%08x\n",
    ↪ fs->bs.extended_bpb.volume_id);
40     DBG_PRINT("[ fat12          ] Volume Label: %.11s\n",
    ↪ fs->bs.extended_bpb.volume_label);
41     DBG_PRINT("[ fat12          ] File System Type: %.8s\n",
    ↪ fs->bs.extended_bpb.file_system_type);
42     DBG_PRINT("[ fat12          ] Signature: 0x%04x\n", fs->bs.signature);
43
44     fs->fat_lba = fs->bs.bpb.reserved_sector_count;
45     fs->fat_sectors = fs->bs.bpb.num_fats * fs->bs.bpb.fat_size_16;
46     fs->root_dir_lba = fs->fat_lba + fs->fat_sectors;
47     fs->root_dir_sectors = ((fs->bs.bpb.root_entry_count *
    ↪ DIRECTORY_ENTRY_SIZE) + (fs->bs.bpb.bytes_per_sector - 1)) /
    ↪ fs->bs.bpb.bytes_per_sector;
48     fs->data_lba = fs->root_dir_lba + fs->root_dir_sectors;
49
50     /* --- NEW --- */
51     fs->cluster_bytes = fs->bs.bpb.sectors_per_cluster *
    ↪ fs->bs.bpb.bytes_per_sector;
52
53     DBG_PRINT("[ fat12          ] FAT start LBA: %u\n", fs->fat_lba);
54     DBG_PRINT("[ fat12          ] FAT size (sectors): %u\n", fs->fat_sectors);
55     DBG_PRINT("[ fat12          ] Root dir start LBA: %u\n", fs->root_dir_lba);
56     DBG_PRINT("[ fat12          ] Root dir size (sectors): %u\n",
    ↪ fs->root_dir_sectors);
57     DBG_PRINT("[ fat12          ] Data region start LBA: %u\n", fs->data_lba);
58
59     /* --- NEW --- */
60     DBG_PRINT("[ fat12          ] Cluster size (bytes): %u\n", fs->cluster_bytes);
61
62     return fs;
63 }

```

Resolving a Path

We now know how to turn a `first_cluster` into an LBA and how many bytes a cluster holds, which is everything we need to locate a file's first cluster and read its data. On top of that, we want the interface every C programmer already knows from POSIX: open a file by its path, read from it, close it. But at this point, all a caller has is a path, not a directory entry. We could make callers search the directory for the entry themselves, but that would turn path walking into their job, when it belongs to the filesystem's internals. So before we can open anything, we need one more piece: `resolve_path`, which turns a path like `/HELLO.TXT` into its directory entry, the one carrying the `first_cluster` we now know how to follow.

We already have `read_root_directory` from Chapter 3, which loads every entry of the root directory. For now, that is all the searching we need: `resolve_path` only looks inside the root directory, so a subdirectory path like `/MYDIR/FILE.TXT` cannot be resolved yet (Chapter 6 retrofits full path walking through nested directories).

One piece is still missing: finding an entry inside a directory by its name. We walk every entry, decode its 8.3 name, and compare it against the name from our path. And there is a gotcha hiding in that comparison: FAT12 names are case-insensitive, so `/hello.txt` has to find `HELLO.TXT`. Names on disk are always uppercase, so we uppercase the name from our path before comparing:

src/fat12.c

```

1  #include <ctype.h>
2
3  static void str_upper(char* s)
4  {
5      // char may be signed, so bytes >= 0x80 could arrive as negative values,
6      // which toupper does not accept; the cast keeps them in 0-255
7      for (int i = 0; s[i] != '\0'; i++) {
8          s[i] = (char)toupper((unsigned char)s[i]);
9      }
10 }
11
12 static DirectoryEntry* find_entry_by_name(
13     DirectoryEntry* entries,
14     uint32_t count,
15     const char* name
16 )
17 {
18     /* Uppercase the name for comparison: names on disk are always uppercase */
19     char target[13];
20     strncpy(target, name, 12);
21     target[12] = '\0';
22     str_upper(target);
23
24     uint32_t offset = 0;
25     DirectoryEntry* entry;
26
27     while ((entry = next_valid_entry(entries, count, &offset)) != NULL)
28     {
29         /* Skip the volume label: it has a name, but it is not a file or
30          ↪ directory */
31         if (entry->attributes & FAT12_ATTR_VOLUME_ID) continue;

```

```

32     char entry_name[13];
33     decode_8_3_name(entry, entry_name);
34
35     if (strcmp(entry_name, target) == 0) return entry;
36 }
37
38 return NULL;
39 }

```

`find_entry_by_name` simply walks every directory entry with `next_valid_entry`, which we already built in Chapter 3. That walk also hands us the volume label, so we skip it here. Every other entry gets its 8.3 name decoded and compared against `target`, the name we are searching for, uppercased so it always matches the case on disk. That is what makes the search case-insensitive. If an entry matches, we return it; if none does, we return `NULL`.

Finally, `resolve_path` ties it together. It rejects anything that is not an absolute path, then treats everything after the leading slash as a single name directly inside the root directory.

src/fat12.c

```

1  static int resolve_path(
2      FAT12FS* fs,
3      const char* path,
4      DirectoryEntry* out
5  )
6  {
7      /* Only absolute paths are supported: every path must start at "/" */
8      if (path[0] != '/') return -1;
9
10     DirectoryEntry* entries = read_root_directory(fs);
11
12     DirectoryEntry* entry = find_entry_by_name(
13         entries,
14         fs->bs.bpb.root_entry_count,
15         path + 1 // skip the leading "/"
16     );
17
18     if (entry != NULL)
19     {
20         memcpy(out, entry, sizeof(DirectoryEntry));
21     }
22
23     free(entries);
24

```

```
25     return entry != NULL ? 0 : -1;
26 }
```

Putting It All Together

We have all the building blocks, so let's declare the public API our filesystem exposes: the open-read-close interface we promised, with every bit of the internal navigation hidden behind it:

include/fat12.h

```
1  typedef struct File File;
2
3  File* fat12_open(FAT12FS* fs, const char* path, char mode);
4  uint32_t fat12_read(File* file, void* buffer, uint32_t size);
5  int fat12_close(File* file);
```

`fat12_open` takes a mode. For now we only support 'r' for reading, but Chapter 7 adds 'w' for writing, and taking the mode now means the signature will not have to change when it does.

Just like `Directory` in Chapter 3, `File` is an opaque handle: the header only declares that it exists, and what it holds is an implementation detail. Since `fat12_open` is what creates and fills in a `File`, we define the struct and `fat12_open` together:

src/fat12.c

```

1  struct File {
2      uint8_t* data;
3      uint32_t size;
4      uint32_t position;
5      char mode;
6  };
7
8  File* fat12_open(FAT12FS* fs, const char* path, char mode)
9  {
10     DBG_PRINT("[ fat12          ] --- fat12_open(\"%s\\", mode='%c') ---\\n", path,
11         ↪ mode);
12
13     if (mode == 'r')
14     {
15         DirectoryEntry resolved;
16         /* Fail if the path does not resolve to an entry */
17         if (resolve_path(fs, path, &resolved) != 0) return NULL;
18         /* Only files can be opened, not directories */
19         if (resolved.attributes & FAT12_ATTR_DIRECTORY) return NULL;
20
21         uint8_t* data = NULL;
22         uint32_t size = resolved.file_size;
23
24         /* Empty files own no cluster (first_cluster is 0), so there is nothing
25            ↪ to read */
26         if (size > 0)
27         {
28             /* One cluster only: anything past it is cut off for now */
29             if (size > fs->cluster_bytes) size = fs->cluster_bytes;
30
31             DBG_PRINT("[ fat12          ] read cluster %u\\n",
32                 ↪ resolved.first_cluster);
33             data = (uint8_t*)malloc(fs->cluster_bytes);
34             block_device_read(fs->device, data_cluster_to_lba(fs,
35                 ↪ resolved.first_cluster), fs->bs.bpb.sectors_per_cluster, data);
36         }
37
38         File* file = (File*)malloc(sizeof(File));
39         file->data = data;
40         file->size = size;
41         file->position = 0;
42         file->mode = 'r';
43         return file;
44     }
45
46     return NULL;
47 }

```

For now, `fat12_open` only accepts `'r'`; any other mode returns `NULL`. We resolve the path and make sure the entry is a file, not a directory, failing

otherwise. If the file is not empty, we read its contents into memory, but only the first cluster for now: we do not know yet where the others live, so a file bigger than one cluster gets cut off there, with size capped to match (Chapter 5 follows the rest of the cluster chain). Empty files are skipped: they own no cluster (their `first_cluster` is 0), so there is nothing to read. Finally, we store the data, its size, the mode, and a starting position of 0 in a new `File` and return it.

Production Note: For simplicity, we load the file's data in `fat12_open`, so `fat12_read` is just a `memcpy` with no disk I/O. A production driver reads clusters on demand inside `read` instead, keeping only a small window of the file in memory. That needs more bookkeeping but handles large files without exhausting memory.

With the heavy lifting done in `fat12_open`, reading is easy: `fat12_read` just copies bytes out of the buffer we already loaded:

src/fat12.c

```

1  uint32_t fat12_read(File* file, void* buffer, uint32_t size)
2  {
3      if (file->mode != 'r') return 0;
4      /* Empty files have no data to copy from */
5      if (file->data == NULL) return 0;
6
7      uint32_t remaining = file->size - file->position;
8      if (size < remaining) remaining = size;
9
10     memcpy(buffer, file->data + file->position, remaining);
11     file->position += remaining;
12
13     return remaining;
14 }
```

`fat12_read` starts with two guards: it only reads files opened in `'r'` mode, and it returns 0 right away for an empty file, which has no data to copy from. Then it copies the caller's size or the bytes left in the file, whichever is smaller (0 once we reach the end), from the current position into buffer, advances

position past them, and returns how many bytes it copied. A return value smaller than size means we hit the end of the file.

All that is left is closing the file:

src/fat12.c

```

1  int fat12_close(File* file)
2  {
3      DBG_PRINT("[ fat12          ] --- fat12_close() ---\n");
4
5      /* free(NULL) is a no-op, so empty files are fine */
6      free(file->data);
7      free(file);
8
9      return 0;
10 }
```

fat12_close frees the data buffer, then the File itself.

The cat Command

With fat12_open, fat12_read, and fat12_close in place, cat almost writes itself. We open the file at the given path, read and print it chunk by chunk until fat12_read returns 0, then close it:

platform/cli/main.c

```

1  static int cmd_cat(FAT12FS *fs, int argc, char *argv[])
2  {
3      if (argc != 3)
4      {
5          fprintf(stderr, "usage: cat <path>\n");
6          return 1;
7      }
8
9      const char *path = argv[2];
10     File *file = fat12_open(fs, path, 'r');
11     if (file == NULL)
12     {
13         fprintf(stderr, "error: file not found\n");
14         return 1;
15     }
```

```

15     }
16
17     uint8_t buf[512];
18     uint32_t bytes;
19     while ((bytes = fat12_read(file, buf, sizeof(buf))) > 0) {
20         fwrite(buf, 1, bytes, stdout);
21     }
22
23     fat12_close(file);
24     return 0;
25 }

```

The 512-byte buffer is an arbitrary choice: `fat12_read` copies as many bytes as we ask for, so the caller never needs to know how big a sector or cluster is.

`main` does not know about `cmd_cat` yet, so we give it one more branch in the dispatch chain, right after `ls`:

platform/cli/main.c

```

1     // ...
2     if (strcmp(command, "ls") == 0) {
3         ret = cmd_ls(fs, argc, argv);
4         /* --- NEW --- */
5     } else if (strcmp(command, "cat") == 0) {
6         ret = cmd_cat(fs, argc, argv);
7         /* --- END NEW --- */
8     } else {
9         fprintf(stderr, "unknown command: %s\n", command);
10        ret = 1;
11    }
12    // ...

```

Time to see it in action. We build with debug output enabled, then `cat HELLO.TXT`, the file we followed by hand earlier in this chapter:

```

1  DEBUG=1 ./build.sh
2  ./fat12-cli cat /HELLO.TXT

```

After the familiar mount output, the trace shows our code retracing the steps we took by hand: `fat12_open` reads the root directory to find the entry, turns its `first_cluster` into an LBA, and reads that one cluster. Then the file's contents appear on screen:

```
1 [ fat12          ] --- fat12_open("/HELLO.TXT", mode='r') ---
2 [ fat12          ] read root directory
3 [ block_device   ] read 32 sector(s) starting at LBA 25
4 [ file_io        ] read 16384 bytes at 0x3200
5 [ fat12          ] read cluster 3
6 [ block_device   ] read 2 sector(s) starting at LBA 59
7 [ file_io        ] read 1024 bytes at 0x7600
8 Hello from FAT12!
9 [ fat12          ] --- fat12_close() ---
```

There it is, sandwiched between the debug lines: `Hello from FAT12!`, the very text we wrote into `HELLO.TXT` back in Chapter 3. Our own code just read a file off the volume. Now compare the trace with the values you worked out by hand earlier: the cluster number, its start LBA and sector count, the byte offset, and the number of bytes read should all match exactly.

What's Next

`first_cluster` is not a mystery anymore: we followed it from a directory entry to a file's bytes, first by hand, then in code. But every file we have read so far fits into a single cluster. Chapter 5 feeds our cat a file that does not, and meets the structure the whole filesystem is named after: the File Allocation Table.

Chapter 5: Reading Multi-Cluster Files

Our `cat` command from Chapter 4 reads `HELL0.TXT` perfectly, but only because its 18 bytes fit into a single cluster. Anything bigger gets cut off after the first one. Time to fix that, and to meet the structure the whole filesystem is named after along the way.

By the end of this chapter, `cat` will print every byte of any file on the volume, following it from cluster to cluster, whatever its size.

When One Cluster Isn't Enough

Before we fix the cut-off, let's watch it happen. First we need a file big enough to trigger it. Our disk image has 2 sectors per cluster and 512 bytes per sector, the values `create_disk.sh` passed to `mkfs.fat` in Chapter 1 (if yours differ, use yours), so each cluster holds $2 * 512 = 1024$ bytes. A cluster is the smallest unit FAT12 allocates, so a file of just 1025 bytes already needs two clusters, even though the second one holds only a single byte.

Let's create exactly that file on our disk image, with the same loop-mount steps we used in Chapter 3:

```
1  sudo mkdir -p /mnt/fat12
2  sudo mount -o loop disk.img /mnt/fat12
3
4  # 1024 'A's for the first cluster, one 'B' for the second
5  { printf '%1024s' '' | tr ' ' 'A'; printf 'B'; } | sudo tee
   ↪ /mnt/fat12/BIGFILE.TXT > /dev/null
6
7  sudo umount /mnt/fat12
```

That gives us `BIGFILE.TXT`, 1025 bytes long: 1024 A's filling its first cluster and a lone B at the start of its second. Let's read it back using our `cat` command:

```
1 ./build.sh
2 ./fat12-cli cat /BIGFILE.TXT
```

```
1 AAAAAAAAAA... (1024 A's)
```

There it is: 1024 A's, and no B. (The shell prompt landing right behind the last A is expected: `BIGFILE.TXT` has no trailing newline.) Our code stopped exactly where Chapter 4 told it to, after the first cluster.

`BIGFILE.TXT` takes two clusters, but its directory entry names only one: `first_cluster`. The obvious guess is that the second one sits right behind it, but nothing guarantees that: a file's cluster-sized pieces do not have to be next to each other, since FAT12 is free to scatter them across the data region.

Why not keep a file's clusters back to back? Because files change. When a file grows, the cluster right after its last one may already belong to another file, and deleted files leave gaps behind that a new file may not fit into in one piece. Letting a file's clusters scatter means any free cluster, wherever it sits, can take its next piece.

Now we have a new problem: how do we know which clusters belong to which file, and in which order do they need to be assembled?

Chaining Clusters with the File Allocation Table

To answer that, FAT12 needs a map that strings each file's clusters together in the right order, starting from the `first_cluster` we already know. That map is the **File Allocation Table**, or FAT: the structure that gave the whole filesystem its name.

It lives in the FAT region, the one region of our volume we have not looked inside yet:

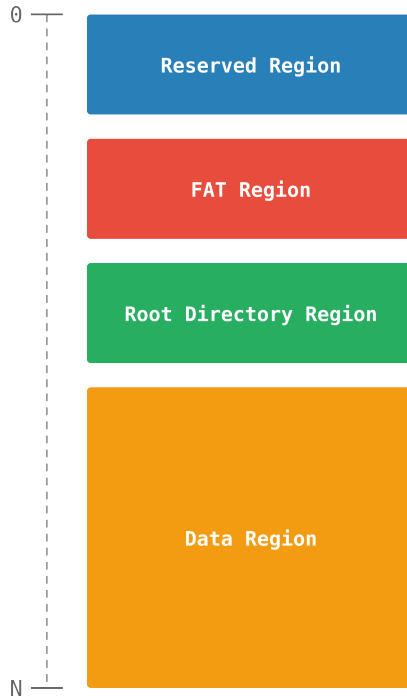


Figure 12. FAT12 volume layout – full

We already know where it sits and how big it is, since Chapter 3 worked out both to find the root directory right behind it. Now we finally need what's inside.

The FAT region holds one or more copies of the FAT itself, almost always two, for redundancy: if a bad sector corrupts one copy, the other survives at a different spot on the volume. That is why Chapter 3 sized the region as $\text{Number of FATs} * \text{FAT Size (sectors)}$. Each copy is $\text{FAT Size (sectors)}$ long, and the copies sit back to back.

Why is only the FAT duplicated? Because it is the one structure every file on the volume depends on. A bad sector in a directory loses the entries stored in that sector, and a bad data cluster damages part of one file. But a single FAT sector holds the links for hundreds of clusters, so losing it breaks every file whose chain runs through them and scrambles which clusters are free. The FAT is also small, so a second copy costs next to nothing. FAT32 later added a backup boot sector as well, but FAT12 and FAT16 keep a backup copy

of nothing except the FAT.

The FAT is a flat array of entries, one per cluster. Each entry is 12 bits wide (that is where the 12 in FAT12 comes from). Twelve bits is exactly one and a half bytes, an odd size for something so fundamental (we will see shortly why that matters). Each cluster has an entry in the FAT that stores the number of the next cluster in the chain. The last cluster's entry holds an end-of-chain marker. Walking this chain reassembles the file's data in order. Here is a made-up example for a file starting at cluster 2, whose chain runs 2 -> 7 -> 4 -> End:

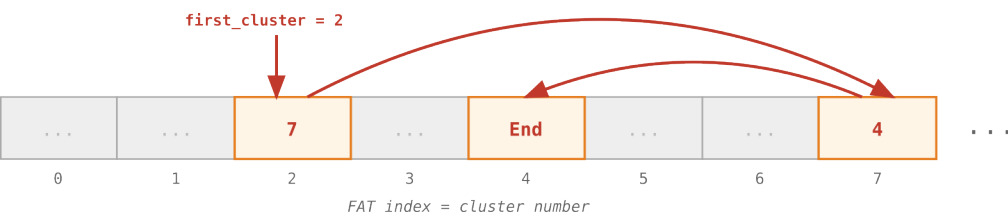


Figure 13. FAT array: walking a cluster chain from first_cluster to end-of-chain

Walking the chain is a simple loop: start at first_cluster, look up that cluster's entry in the FAT, jump to the cluster it names, and repeat until an entry holds the end-of-chain marker.

With the chain in hand, all that is left is to read each of those clusters from the data region and append them in exactly that order:

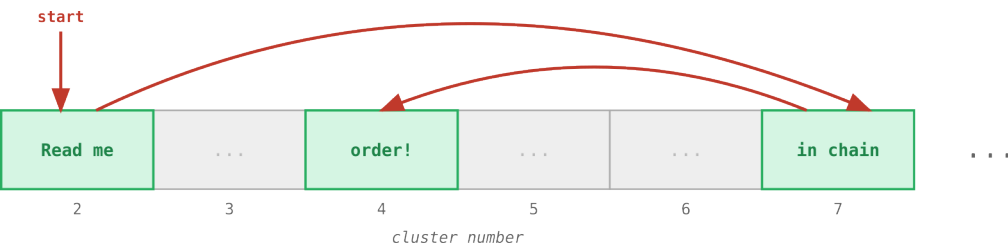


Figure 14. Data region: the same chain followed through the actual clusters

Put together, the three pieces give us the file's complete content:

1 Read me in chain order!



This scattering has a name, *fragmentation*, and it is what a defragmenter cleans up: it moves clusters around until each file’s clusters follow one another in order, then rewrites the FAT and the directory entries to match. The payoff is speed on spinning hard disks. A contiguous file comes off the platter in one long sequential read, while a scattered one sends the drive’s head seeking back and forth, and every seek costs milliseconds. SSDs and flash memory have no moving parts, so reaching a cluster costs about the same wherever it sits, and defragmenting them buys little beyond extra wear.

Not every FAT entry holds the number of a next cluster, though. A handful of specific values carry special meaning instead:

Value (hex)	Name	Meaning
0x000	Free cluster	Available for allocation
0xFF7	Bad cluster	Physically damaged, so the filesystem skips it permanently (found by formatting/repair tools; modern drives remap bad sectors in their own firmware, so the filesystem rarely sees one today)
0xFF8-0xFFF	End-of-chain marker	Last cluster in a file’s chain. Microsoft’s FAT drivers write 0xFFF, and so does Linux’s kernel FAT driver. The spec notes that some disk utilities use other values from the range, so a conformant reader tests the whole range ($\geq 0xFF8$) on read

The first two FAT entries, `FAT[0]` and `FAT[1]`, are reserved and never hold a chain link. Since cluster `N`'s link lives in `FAT[N]`, no cluster can be numbered 0 or 1, and that is why the first cluster of the data region is cluster 2. `FAT[0]` repeats the BPB's media descriptor byte in its low 8 bits with all other bits set to 1 (0xFF8 on our volume), and `FAT[1]` always holds an end-of-chain marker, 0xFFFF.

Why repeat the media descriptor? It is a duplicate, just like the volume label from Chapter 3 that lives in both the boot sector and the root directory, and a legacy field our code never reads. Appendix C: The Boot Sector's Leftovers explains where it comes from.

Now we know everything a FAT entry can hold. Time for the hard part: actually pulling one out of the raw bytes for a given cluster. It is simple in principle, but here that odd 12-bit stride finally catches up with us, and it teams up with little-endian byte order: decoding an entry takes more care than anything we have done so far.

Decoding the 12-Bit Entries

Time to get our hands dirty again, this time with `BIGFILE.TXT`. First we need its first cluster, so let's peek inside the root directory, just as we did in Chapter 4. Its byte offset still comes from the `Root dir start LBA` and `Bytes Per Sector` values in our debug output:

```
1 Byte offset = Root dir start LBA * Bytes per sector
```

Replace `<offset>` with your calculated byte offset. This time we ask for 160 bytes instead of 128, one more 32-byte entry, so `BIGFILE.TXT` shows up and we can read its size and first cluster:

```
1 xxd -g 1 -s <offset> -l 160 disk.img
```

```

1  00003200: 4d 59 44 49 53 4b 20 20 20 20 20 08 00 00 51 7b MYDISK      ...Q{
2  00003210: 3c 5d 3c 5d 00 00 51 7b 3c 5d 00 00 00 00 00 00 <]<]..Q{<].....
3  00003220: 48 45 4c 4c 4f 20 20 20 54 58 54 20 00 98 61 7d HELLO    TXT ..a}
4  00003230: 3d 5d 3d 5d 00 00 61 7d 3d 5d 03 00 12 00 00 00 =]=]..a}=].....
5  00003240: 52 45 41 44 4d 45 20 20 54 58 54 20 00 67 64 7d README  TXT .gd}
6  00003250: 3d 5d 3d 5d 00 00 64 7d 3d 5d 04 00 0f 00 00 00 =]=]..d}=].....
7  00003260: 4d 59 44 49 52 20 20 20 20 20 20 10 00 91 66 7d MYDIR      ...f}
8  00003270: 3d 5d 3d 5d 00 00 66 7d 3d 5d 05 00 00 00 00 00 =]=]..f}=].....
9  00003280: 42 49 47 46 49 4c 45 20 54 58 54 20 00 16 53 90 BIGFILE  TXT ..S.
10 00003290: 43 5d 43 5d 00 00 53 90 43 5d 06 00 01 04 00 00 C]C]..S.C].....

```

Same trick as in Chapter 4: focus on `BIGFILE.TXT`'s entry, the last two rows. Its last six bytes hold `first_cluster` (2 bytes) and `file_size` (4 bytes), both little-endian, so we read their bytes back to front. (Your dump may show different values; if so, use yours.)

```

1  first_cluster:  06 00      -> 0x0006      =  6
2  file_size:      01 04 00 00 -> 0x00000401 = 1025

```

`BIGFILE.TXT` starts at cluster 6. To find out which cluster comes next, we need cluster 6's FAT entry, and for that we first need to know where the FAT itself starts. Let's build and run with debug output enabled to get `FAT start LBA` and `Bytes Per Sector`:

```

1  DEBUG=1 ./build.sh
2  ./fat12-cli ls /

```

Among the output are the two lines we need:

```

1  [ fat12      ] Bytes Per Sector: 512
2  [ fat12      ] FAT start LBA: 1

```

So the FAT starts at byte offset `FAT start LBA * Bytes Per Sector`. Replace `<fat_offset>` with your calculated value:

```

1  xxd -g 1 -s <fat_offset> -l 16 disk.img

```

```
1 00000200: f8 ff ff 00 f0 ff ff ff ff 07 f0 ff 00 00 00 00 .....
```

We can already spot something familiar: the very first byte, `f8`, is our media descriptor, and together with the next two bytes it makes up the two reserved entries, `FAT[0]` and `FAT[1]`. Beyond that, reading the dump gets hard. With 12 bits per entry, the entries do not line up with byte boundaries the way everything else has so far, so nothing in the dump marks where one entry ends and the next begins. This is exactly where most people implementing FAT12 get stuck, and the source of countless headaches. Don't worry: we will take our time and work through it step by step.

Think of the entries as sliding windows across the byte array: cluster 0's entry covers byte 0 plus half of byte 1 (half a byte is also called a *nibble*), cluster 1's entry covers the other half of byte 1 plus byte 2, and so on. Which half goes where is a detail we will sort out shortly.

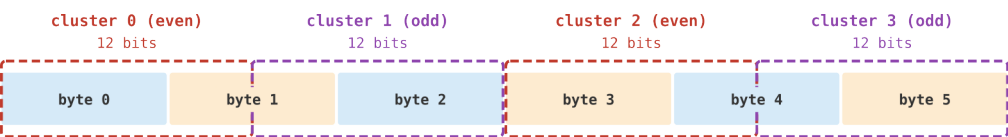


Figure 15. 12-bit entries as 1.5-byte sliding windows across the FAT byte array

As the diagram shows, entry boundaries do not line up with byte boundaries. Every entry spans two bytes, and one byte in every three is split down the middle between two neighboring entries, half for each. Which half an entry gets depends on whether its cluster number is even or odd. Let's zoom in one level further and split every byte into its two nibbles:

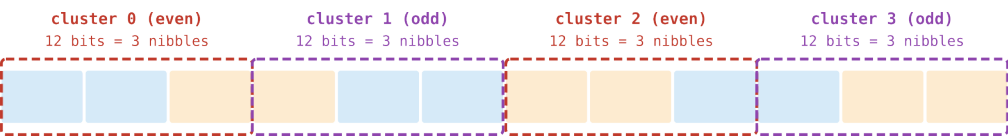


Figure 16. The same windows with every byte split into two nibbles

Now everything lines up: each entry is exactly three nibbles, and every boundary between two entries falls on a nibble boundary.

Now for the tricky part. Like every multi-byte value in FAT12, the FAT is stored little-endian: whenever two bytes form one value, the first byte is

the least significant one. Each entry is 12 bits of such a two-byte value. We already ran into this when we read `file_size` and `first_cluster` out of `BIGFILE.TXT`'s directory entry earlier in this chapter and had to put their bytes together in reverse order. Inside a single byte, though, nothing is reversed: its two nibbles always read most significant first, exactly like the two hex digits `xxd` prints (in `f8`, the `f` is the high nibble). Reversed bytes around unreversed nibbles lead to a surprising twist. In every shared byte, the two halves go the opposite way from what the diagrams above suggest: the even entry gets the *low* nibble on the right, and the odd entry gets the *high* nibble on the left. Each entry ends up with a gap in the middle:

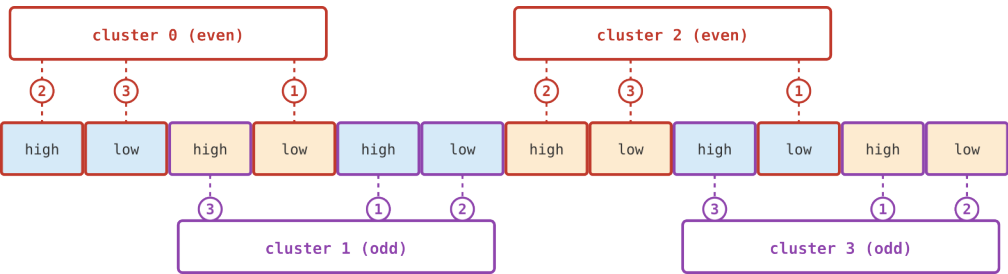


Figure 17. The real layout: in each shared byte, the even entry owns the low nibble and the odd entry the high nibble

The numbers on the lines give each nibble's place in the entry's value, 1 being the most significant. Little-endian puts the second byte in front, so an entry's value never reads left to right across the bytes.

Let's see this on our real FAT. We already have its bytes from the dump above, and since every hex digit is exactly one nibble, the hex dump shows us every nibble directly. Bytes 6 to 11, `ff ff ff 07 f0 ff`, hold clusters 4 to 7 and with them `BIGFILE.TXT`'s chain. Here they are, laid out exactly like the previous diagram, one hex digit per nibble:

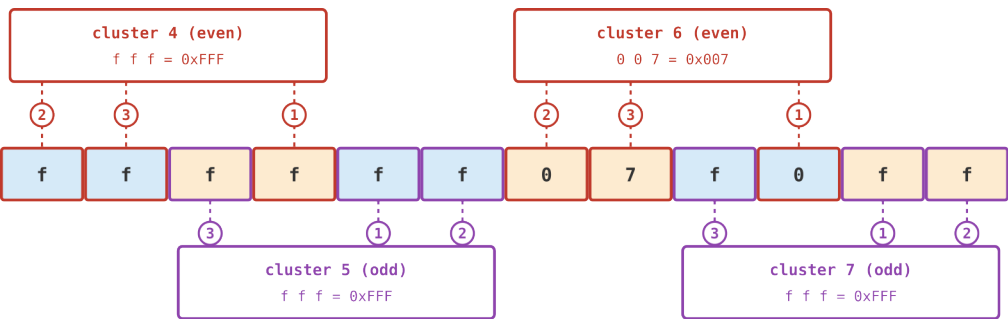


Figure 18. Decoding clusters 4 to 7 from FAT bytes 6 to 11

Reading an entry takes three steps, and only the last one differs between even and odd clusters. First, find the byte where the entry begins. Every entry takes one and a half bytes, so cluster N's entry starts $N * 1.5$ bytes into the FAT. For an odd cluster that lands in the middle of a byte, so we round down to the byte holding its first nibble:

1 $\text{Byte offset} = \text{Cluster} + \text{Cluster} / 2 \quad (\text{rounded down})$

Let's follow `BIGFILE.TXT`'s chain through all three steps, keeping the diagram in view. It starts at cluster 6, and applying the formula puts cluster 6's entry at $6 + 6 / 2 = 6 + 3 = 9$.

Second, read two bytes from there: bytes 9 and 10. Third, line up the entry's three nibbles, most significant first. Little-endian puts the second byte's nibbles in front, and inside each byte the nibbles keep their order. Following the numbered lines, 1 to 3:

1 Cluster 6 (even): `0 0 7 = 0x007`

Cluster 6 points to cluster 7. Same three steps again: cluster 7's entry starts at $7 + 7 / 2 = 7 + 3 = 10$ ($7 / 2$ is 3.5, rounded down to 3), so we read bytes 10 and 11:

1 Cluster 7 (odd): `f f f = 0xFFF`

Cluster 7 holds the end-of-chain marker. `BIGFILE.TXT`'s complete chain is `6 -> 7 -> End`, decoded entirely by hand.

One last check before we write code: if the chain is right, cluster 7 holds the B that our code could not reach at the start of this chapter. We find it with the same two formulas as in Chapter 4:

```
1 Cluster LBA = Data region start LBA + (Cluster - 2) * Sectors per cluster
2 Byte offset = Cluster LBA * Bytes per sector
```

Replace <offset> with your calculated byte offset. 16 bytes are enough to see what the cluster holds:

```
1 xxd -g 1 -s <offset> -l 16 disk.img

1 00008600: 42 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  B.....
```

There it is: our lone B, followed by the zeros of the cluster's slack space. The FAT sent us to the right place.

Now let's teach the computer to do what we just did with xxd and a pencil. It will decode entries from a copy of the FAT held in memory, so FAT12FS needs a pointer to one. Almost every operation we build from here on looks up FAT entries, often many of them, so we read the FAT from disk just once and keep it in memory for as long as the volume is mounted. That avoids hundreds of tiny disk reads, and since the FAT is small, the memory cost is negligible. We add the field now, but leave it empty for the moment: loading the FAT into it is the next section's job, and for now we stay focused on decoding:

src/fat12.c

```

1  struct FAT12FS {
2      BlockDevice* device;
3      BootSector bs;
4      uint32_t fat_lba;
5      uint32_t fat_sectors;
6      uint32_t root_dir_lba;
7      uint32_t root_dir_sectors;
8      uint32_t data_lba;
9      uint32_t cluster_bytes;
10
11     /* --- NEW --- */
12     uint8_t* fat;
13 };

```

Now for the function that does the actual decoding. `get_fat_entry` takes a cluster number and returns the value stored in its FAT entry, handling even and odd clusters exactly the way we just did by hand:

src/fat12.c

```

1  static uint16_t get_fat_entry(
2      FAT12FS* fs,
3      uint16_t cluster
4  )
5  {
6      /* byte where this cluster's 12-bit entry starts (1.5 bytes per entry);
7       integer division drops the .5 for odd clusters: 7 / 2 = 3 */
8      uint32_t offset = cluster + (cluster / 2);
9
10     /* read the two bytes the entry spans as one little-endian 16-bit value
11      (second byte on top); this closes the gap: on-disk bytes 07 f0 become
12      0xF007, the entry's three nibbles side by side and the neighbor's
13      nibble at one end, so all that is left is to cut that one off */
14     uint16_t value = (fs->fat[offset + 1] << 8) | fs->fat[offset];
15
16     /* lowest bit set = odd cluster, clear = even */
17     if (cluster & 1)
18     {
19         /* odd cluster: drop the last nibble (it belongs to the even entry) */
20         return value >> 4;
21     }
22     else
23     {
24         /* even cluster: clear the first nibble (it belongs to the odd entry) */
25         return value & 0xFFFF;
26     }
27 }

```

First, `offset` finds the byte where the entry begins, with the same formula we used by hand. Next, `value` combines that byte and the one after it into a single 16-bit value in little-endian order: `<< 8` moves the second byte up into the most significant half, and `|` drops the first byte in as the least significant half. That alone closes the gap: the entry's three nibbles now sit side by side, and the neighbor's nibble sits at one end.

All that is left is to drop that one nibble, and whether the cluster is odd or even tells us which end it is on. For an odd cluster it is the last nibble, so `>> 4` shifts the value right by 4 bits, exactly one nibble, and pushes it out. For an even cluster it is the first nibble, so `& 0x0FFF` masks it off: each hex digit of the mask covers one nibble, the `0` clears it, and the three `F`s keep the rest. Here are both of `BIGFILE.TXT`'s entries:

```
1 Cluster 6 (even), bytes 9 and 10:  07 f0  ->  0xf007  & 0x0fff  ->  0x007
2 Cluster 7 (odd), bytes 10 and 11:  f0 ff  ->  0xffff0 >> 4      ->  0xfff
```

Take a breath here. This is the trickiest part of FAT12, and almost nobody gets it on the first pass. If the gap, the swapped nibbles, or the mask and shift still feel slippery, go back to the diagrams and follow the numbered lines again, as many times as it takes. It is worth it: once 12-bit packing clicks, it stays clicked. Want to test yourself? Look up `HELLO.TXT`'s first cluster in the root directory dump, then decode that cluster's FAT entry from the FAT dump. You should land on the end-of-chain marker, `0xFFF`.

Loading the FAT

In the previous section we made room for the FAT in `FAT12FS` but left it empty, so we could focus on decoding the 12-bit entries. Filling it is the easy part. We already know where the FAT starts, `FAT_start_LBA` from Chapter 3, and how big one copy is, the BPB's `FAT_Size (sectors)`. Let's load its first copy on mount:

src/fat12.c

```

1  FAT12FS* fat12_mount(BlockDevice* device)
2  {
3      DBG_PRINT("[ fat12      ] --- fat12_mount() ---\n");
4      FAT12FS* fs = malloc(sizeof(FAT12FS));
5      fs->device = device;
6      fs->bs = read_boot_sector(device);
7
8      /* jump, boot_code, reserved_nt: not used by FAT itself, so we skip them */
9      DBG_PRINT("[ fat12      ] OEM Name: %.8s\n", fs->bs.oem_name);
10     DBG_PRINT("[ fat12      ] Bytes Per Sector: %u\n",
11         ↪ fs->bs.bpb.bytes_per_sector);
12     DBG_PRINT("[ fat12      ] Sectors Per Cluster: %u\n",
13         ↪ fs->bs.bpb.sectors_per_cluster);
14     DBG_PRINT("[ fat12      ] Reserved Sector Count: %u\n",
15         ↪ fs->bs.bpb.reserved_sector_count);
16     DBG_PRINT("[ fat12      ] Number of FATs: %u\n", fs->bs.bpb.num_fats);
17     DBG_PRINT("[ fat12      ] Root Entry Count: %u\n",
18         ↪ fs->bs.bpb.root_entry_count);
19     DBG_PRINT("[ fat12      ] Total Sectors (16): %u\n",
20         ↪ fs->bs.bpb.total_sectors_16);
21     DBG_PRINT("[ fat12      ] Media Descriptor: 0x%02x\n", fs->bs.bpb.media);
22     DBG_PRINT("[ fat12      ] FAT Size (sectors): %u\n",
23         ↪ fs->bs.bpb.fat_size_16);
24     DBG_PRINT("[ fat12      ] Sectors Per Track: %u\n",
25         ↪ fs->bs.bpb.sectors_per_track);
26     DBG_PRINT("[ fat12      ] Number of Heads: %u\n",
27         ↪ fs->bs.bpb.number_of_heads);
28     DBG_PRINT("[ fat12      ] Hidden Sectors: %u\n",
29         ↪ fs->bs.bpb.hidden_sectors);
30     DBG_PRINT("[ fat12      ] Total Sectors (32): %u\n",
31         ↪ fs->bs.bpb.total_sectors_32);
32     DBG_PRINT("[ fat12      ] Drive Number: 0x%02x\n",
33         ↪ fs->bs.extended_bpb.drive_number);
34     DBG_PRINT("[ fat12      ] Boot Signature: 0x%02x\n",
35         ↪ fs->bs.extended_bpb.boot_signature);
36     DBG_PRINT("[ fat12      ] Volume ID: 0x%08x\n",
37         ↪ fs->bs.extended_bpb.volume_id);
38     DBG_PRINT("[ fat12      ] Volume Label: %.11s\n",
39         ↪ fs->bs.extended_bpb.volume_label);
40     DBG_PRINT("[ fat12      ] File System Type: %.8s\n",
41         ↪ fs->bs.extended_bpb.file_system_type);
42     DBG_PRINT("[ fat12      ] Signature: 0x%04x\n", fs->bs.signature);
43
44     fs->fat_lba = fs->bs.bpb.reserved_sector_count;
45     fs->fat_sectors = fs->bs.bpb.num_fats * fs->bs.bpb.fat_size_16;
46     fs->root_dir_lba = fs->fat_lba + fs->fat_sectors;
47     fs->root_dir_sectors = ((fs->bs.bpb.root_entry_count *
48         ↪ DIRECTORY_ENTRY_SIZE) + (fs->bs.bpb.bytes_per_sector - 1)) /
49         ↪ fs->bs.bpb.bytes_per_sector;
50     fs->data_lba = fs->root_dir_lba + fs->root_dir_sectors;

```

```

34     fs->cluster_bytes = fs->bs.bpb.sectors_per_cluster *
        ↳ fs->bs.bpb.bytes_per_sector;
35
36     DBG_PRINT("[ fat12          ] FAT start LBA: %u\n", fs->fat_lba);
37     DBG_PRINT("[ fat12          ] FAT size (sectors): %u\n", fs->fat_sectors);
38     DBG_PRINT("[ fat12          ] Root dir start LBA: %u\n", fs->root_dir_lba);
39     DBG_PRINT("[ fat12          ] Root dir size (sectors): %u\n",
        ↳ fs->root_dir_sectors);
40     DBG_PRINT("[ fat12          ] Data region start LBA: %u\n", fs->data_lba);
41     DBG_PRINT("[ fat12          ] Cluster size (bytes): %u\n", fs->cluster_bytes);
42
43     /* --- NEW --- */
44     DBG_PRINT("[ fat12          ] load FAT (first copy)\n");
45     fs->fat = (uint8_t*)malloc(fs->bs.bpb.fat_size_16 *
        ↳ fs->bs.bpb.bytes_per_sector);
46     block_device_read(device, fs->fat_lba, fs->bs.bpb.fat_size_16, fs->fat);
47     /* --- END NEW --- */
48
49     return fs;
50 }

```

fat12_mount now loads the first copy of the FAT into memory, all that is left is to free that memory again when the volume is unmounted:

src/fat12.c

```

1 void fat12_umount(FAT12FS* fs)
2 {
3     /* --- NEW --- */
4     free(fs->fat);
5
6     free(fs);
7 }

```

Counting the Clusters

Before we implement the cluster chain walk, we need one more number: how many clusters the data region holds. That count is how we tell a real cluster number from anything else, whether it is one of the special FAT values we just met or a corrupted directory entry claiming a file starts at cluster 9999 on a volume with only about 4000 data clusters. It is also how our chain walk will know when to stop.

Every sector from the data region's start to the end of the volume belongs to the data region. The volume's total sector count comes from the BPB, split across two fields: `Total Sectors (16)` holds it when the volume fits in 16 bits, and `Total Sectors (32)` takes over once it doesn't, with whichever field is unused left at zero. Subtracting the data region's start from whichever one is set gives us the size of the data region in sectors. Dividing that by `Sectors Per Cluster` turns it into a cluster count:

- 1 `Total data sectors = Total sectors - Data region start LBA`
- 2 `Data cluster count = Total data sectors / Sectors per cluster`

The division rounds down: when the data region's sector count is not an exact multiple of `Sectors Per Cluster`, the sectors left over at the end are fewer than one cluster's worth, so they simply go unused.



Why two fields for the same value? Because the original 16-bit field ran out: once disks grew past its ceiling, `Total Sectors (32)` was added next to it instead of replacing it. It does not make old drivers understand large volumes (one that only checks `Total Sectors (16)` would see 0 and get the size wrong), but it does mean every volume that still fits in 16 bits keeps working exactly as before, since that field is untouched for them.

Let's compute the data cluster count once on mount and cache it in FAT12FS, using whichever total-sectors field is set:

src/fat12.c

```

1  struct FAT12FS {
2      BlockDevice* device;
3      BootSector bs;
4      uint32_t fat_lba;
5      uint32_t fat_sectors;
6      uint32_t root_dir_lba;
7      uint32_t root_dir_sectors;
8      uint32_t data_lba;
9      uint32_t cluster_bytes;
10     uint8_t* fat;
11
12     /* --- NEW --- */
13     uint32_t data_cluster_count;
14 };
15
16 FAT12FS* fat12_mount(BlockDevice* device)
17 {
18     DBG_PRINT("[ fat12      ] --- fat12_mount() ---\n");
19     FAT12FS* fs = malloc(sizeof(FAT12FS));
20     fs->device = device;
21     fs->bs = read_boot_sector(device);
22
23     /* jump, boot_code, reserved_nt: not used by FAT itself, so we skip them */
24     DBG_PRINT("[ fat12      ] OEM Name: %.8s\n", fs->bs.oem_name);
25     DBG_PRINT("[ fat12      ] Bytes Per Sector: %u\n",
26         ↪ fs->bs.bpb.bytes_per_sector);
27     DBG_PRINT("[ fat12      ] Sectors Per Cluster: %u\n",
28         ↪ fs->bs.bpb.sectors_per_cluster);
29     DBG_PRINT("[ fat12      ] Reserved Sector Count: %u\n",
30         ↪ fs->bs.bpb.reserved_sector_count);
31     DBG_PRINT("[ fat12      ] Number of FATs: %u\n", fs->bs.bpb.num_fats);
32     DBG_PRINT("[ fat12      ] Root Entry Count: %u\n",
33         ↪ fs->bs.bpb.root_entry_count);
34     DBG_PRINT("[ fat12      ] Total Sectors (16): %u\n",
35         ↪ fs->bs.bpb.total_sectors_16);
36     DBG_PRINT("[ fat12      ] Media Descriptor: 0x%02x\n", fs->bs.bpb.media);
37     DBG_PRINT("[ fat12      ] FAT Size (sectors): %u\n",
38         ↪ fs->bs.bpb.fat_size_16);
39     DBG_PRINT("[ fat12      ] Sectors Per Track: %u\n",
40         ↪ fs->bs.bpb.sectors_per_track);
41     DBG_PRINT("[ fat12      ] Number of Heads: %u\n",
42         ↪ fs->bs.bpb.number_of_heads);
43     DBG_PRINT("[ fat12      ] Hidden Sectors: %u\n",
44         ↪ fs->bs.bpb.hidden_sectors);
45     DBG_PRINT("[ fat12      ] Total Sectors (32): %u\n",
46         ↪ fs->bs.bpb.total_sectors_32);
47     DBG_PRINT("[ fat12      ] Drive Number: 0x%02x\n",
48         ↪ fs->bs.extended_bpb.drive_number);
49     DBG_PRINT("[ fat12      ] Boot Signature: 0x%02x\n",
50         ↪ fs->bs.extended_bpb.boot_signature);

```

```

39     DBG_PRINT("[ fat12          ] Volume ID: 0x%08x\n",
    ↪ fs->bs.extended_bpb.volume_id);
40     DBG_PRINT("[ fat12          ] Volume Label: %.11s\n",
    ↪ fs->bs.extended_bpb.volume_label);
41     DBG_PRINT("[ fat12          ] File System Type: %.8s\n",
    ↪ fs->bs.extended_bpb.file_system_type);
42     DBG_PRINT("[ fat12          ] Signature: 0x%04x\n", fs->bs.signature);
43
44     fs->fat_lba = fs->bs.bpb.reserved_sector_count;
45     fs->fat_sectors = fs->bs.bpb.num_fats * fs->bs.bpb.fat_size_16;
46     fs->root_dir_lba = fs->fat_lba + fs->fat_sectors;
47     fs->root_dir_sectors = ((fs->bs.bpb.root_entry_count *
    ↪ DIRECTORY_ENTRY_SIZE) + (fs->bs.bpb.bytes_per_sector - 1)) /
    ↪ fs->bs.bpb.bytes_per_sector;
48     fs->data_lba = fs->root_dir_lba + fs->root_dir_sectors;
49     fs->cluster_bytes = fs->bs.bpb.sectors_per_cluster *
    ↪ fs->bs.bpb.bytes_per_sector;
50
51     /* --- NEW --- */
52     uint32_t total_sectors = fs->bs.bpb.total_sectors_16
53         ? fs->bs.bpb.total_sectors_16
54         : fs->bs.bpb.total_sectors_32;
55
56     fs->data_cluster_count = (total_sectors - fs->data_lba) /
    ↪ fs->bs.bpb.sectors_per_cluster;
57     /* --- END NEW --- */
58
59     DBG_PRINT("[ fat12          ] FAT start LBA: %u\n", fs->fat_lba);
60     DBG_PRINT("[ fat12          ] FAT size (sectors): %u\n", fs->fat_sectors);
61     DBG_PRINT("[ fat12          ] Root dir start LBA: %u\n", fs->root_dir_lba);
62     DBG_PRINT("[ fat12          ] Root dir size (sectors): %u\n",
    ↪ fs->root_dir_sectors);
63     DBG_PRINT("[ fat12          ] Data region start LBA: %u\n", fs->data_lba);
64     DBG_PRINT("[ fat12          ] Cluster size (bytes): %u\n", fs->cluster_bytes);
65
66     /* --- NEW --- */
67     DBG_PRINT("[ fat12          ] Data cluster count: %u\n",
    ↪ fs->data_cluster_count);
68     /* --- END NEW --- */
69
70     DBG_PRINT("[ fat12          ] load FAT (first copy)\n");
71     fs->fat = (uint8_t*)malloc(fs->bs.bpb.fat_size_16 *
    ↪ fs->bs.bpb.bytes_per_sector);
72     block_device_read(device, fs->fat_lba, fs->bs.bpb.fat_size_16, fs->fat);
73
74     return fs;
75 }

```

Why clusters instead of sectors? Nothing forces a cluster to span more than one sector: a single sector per cluster is perfectly legal, and high-density floppy disks use exactly that. But 12-bit FAT entries can only number so many clusters, and with one sector per cluster our disk image would need far more than that, so we group two sectors into each cluster. Beyond that, cluster size is a trade-off: bigger clusters mean fewer of them, so a smaller FAT and fewer reads per file, but more slack space wasted at the end of most files, since even a tiny file occupies a whole cluster.

Reading a Cluster Chain

Now we have both pieces the walk needs: we can decode any cluster's FAT entry, and we know how many data clusters the volume holds. Walking a chain is a simple loop: start at the first cluster, read its data, look up its FAT entry to find the next cluster, and keep going as long as that number is a real data cluster, somewhere from the first data cluster (2) up to the volume's last one. The end-of-chain marker lies outside that range, and so does the bad-cluster marker or any number a corrupted entry might hold, so all of them end the walk.

There is one catch: we need a buffer big enough for the whole chain before we can read anything into it. So the chain itself has to tell us how long it is, and that is why we walk it twice: the first pass only counts the chain's clusters, so we can allocate exactly the right buffer, and the second reads every cluster into its slot:

src/fat12.c

```

1  static int is_data_cluster(FAT12FS* fs, uint16_t cluster)
2  {
3      /* real data clusters run from 2 up to the volume's last one; free (0) lies
4         below that range, and since FAT12 caps a volume at 4084 data clusters
5         (highest cluster 0xFF5), bad (0xFF7) and end-of-chain (0xFF8+) lie above
        ↪ */
6      return cluster >= FIRST_DATA_CLUSTER && (uint32_t)(cluster -
        ↪ FIRST_DATA_CLUSTER) < fs->data_cluster_count;
7  }
8
9  static uint8_t* read_cluster_chain(
10     FAT12FS* fs,
11     uint16_t first_cluster,
12     uint32_t* out_bytes
13 )
14 {
15     /* first pass: count the chain's clusters, so we know how much to allocate
        ↪ */
16     uint32_t cluster_count = 0;
17     for (uint16_t cluster = first_cluster; is_data_cluster(fs, cluster);
        ↪ cluster = get_fat_entry(fs, cluster))
18     {
19         /* no chain can be longer than the volume has clusters: if this one is,
20            a corrupt FAT has looped it back onto itself, so give up */
21         if (cluster_count == fs->data_cluster_count)
22             return NULL;
23
24         cluster_count++;
25     }
26
27     uint8_t* data = (uint8_t*)malloc(cluster_count * fs->cluster_bytes);
28
29     /* second pass: the first one already checked these clusters, so we just
30        read all of them, in chain order, each into its slot */
31     uint16_t cluster = first_cluster;
32     for (uint32_t i = 0; i < cluster_count; i++)
33     {
34         DBG_PRINT("[ fat12          ] read cluster %u\n", cluster);
35         block_device_read(fs->device, data_cluster_to_lba(fs, cluster),
        ↪ fs->bs.bpb.sectors_per_cluster, data + i * fs->cluster_bytes);
36         cluster = get_fat_entry(fs, cluster);
37     }
38
39     *out_bytes = cluster_count * fs->cluster_bytes;
40     return data;
41 }

```

Why not one pass that grows the buffer? We could `realloc` a bigger buffer whenever the chain outgrows it, but every grow may copy all the file data read so far, and the loop would need to track a capacity and pick a growth policy on top. A second walk over the in-memory FAT costs far less and keeps the code simpler.

The end-of-chain marker (`0xFFF`, or any value $\geq 0xFF8$) is never a valid data cluster, since even the largest FAT12 volume ends at cluster `0xFF5`, so `is_data_cluster` rejects it and both passes stop there – same for the bad-cluster marker `0xFF7`. One bounds check against the volume's own cluster count catches both, plus outright corruption: a directory entry that claims a file starts at cluster 9999 on a 4000-cluster volume won't send our walk past the data region either.

A corrupt FAT can also link a chain back onto itself (6 -> 7 -> 6). Every cluster number in such a loop passes `is_data_cluster`, so without a limit the first pass would never end. But no chain can be longer than the volume has clusters, so once our count reaches `data_cluster_count`, we know the chain loops and give up, returning `NULL`.

Putting It All Together

We already have `fat12_open` from Chapter 4, reading a single cluster and capping the file's size to match. With `read_cluster_chain` in hand, we can lift that cap: we replace the single-cluster read with a walk of the whole chain. `fat12_read` and `fat12_close` stay exactly as they are:

src/fat12.c

```

1 File* fat12_open(FAT12FS* fs, const char* path, char mode)
2 {
3     DBG_PRINT("[ fat12          ] --- fat12_open(\"%s\", mode='%c') ---\n", path,
4         ↪ mode);
5
6     if (mode == 'r')
7     {
8         DirectoryEntry resolved;
9         /* Fail if the path does not resolve to an entry */
10        if (resolve_path(fs, path, &resolved) != 0) return NULL;
11        /* Only files can be opened, not directories */
12        if (resolved.attributes & FAT12_ATTR_DIRECTORY) return NULL;
13
14        uint8_t* data = NULL;
15
16        /* --- NEW --- */
17        uint32_t chain_bytes = 0;
18
19        if (resolved.file_size > 0)
20        {
21            data = read_cluster_chain(fs, resolved.first_cluster, &chain_bytes);
22            if (data == NULL) return NULL;
23        }
24        /* --- END NEW --- */
25
26        File* file = (File*)malloc(sizeof(File));
27        file->data = data;
28
29        /* --- NEW --- */
30        /* a chain shorter than file_size means a corrupt volume:
31        never let fat12_read run past the bytes we actually loaded */
32        file->size = resolved.file_size < chain_bytes ? resolved.file_size :
33            ↪ chain_bytes;
34        /* --- END NEW --- */
35
36        file->position = 0;
37        file->mode = 'r';
38        return file;
39    }
40    return NULL;
41 }

```

We replace the single-cluster read with a walk of the whole chain, so data now holds every cluster of the file. The file's size still comes from its directory entry, unless the chain runs out first: on a corrupt volume, a stray end-of-chain marker can cut the chain short of file_size, and fat12_read would then copy right past the end of our buffer. Taking the smaller of file_size

and `chain_bytes` turns that into a truncated file instead. On a healthy volume `file_size` always wins, so the slack space at the end of the last cluster never reaches the caller.

Finding the Missing B

Nothing about the API has changed; only its internals have learned to follow a cluster chain, so our `cat` command needs no changes at all. Let's build with debug output enabled and give `BIGFILE.TXT` another try, the file that got cut off after its first cluster at the start of this chapter:

```
1 DEBUG=1 ./build.sh
2 ./fat12-cli cat /BIGFILE.TXT
```

The output should look like this:

```
1 ... (boot sector read and raw BPB fields from Chapter 2, unchanged since then)
2 [ fat12          ] FAT start LBA: 1
3 [ fat12          ] FAT size (sectors): 24
4 [ fat12          ] Root dir start LBA: 25
5 [ fat12          ] Root dir size (sectors): 32
6 [ fat12          ] Data region start LBA: 57
7 [ fat12          ] Cluster size (bytes): 1024
8 [ fat12          ] Data cluster count: 4067
9 [ fat12          ] load FAT (first copy)
10 [ block_device ] read 12 sector(s) starting at LBA 1
11 [ file_io       ] read 6144 bytes at 0x200
12 [ fat12          ] --- fat12_open("/BIGFILE.TXT", mode='r') ---
13 [ fat12          ] read root directory
14 [ block_device ] read 32 sector(s) starting at LBA 25
15 [ file_io       ] read 16384 bytes at 0x3200
16 [ fat12          ] read cluster 6
17 [ block_device ] read 2 sector(s) starting at LBA 65
18 [ file_io       ] read 1024 bytes at 0x8200
19 [ fat12          ] read cluster 7
20 [ block_device ] read 2 sector(s) starting at LBA 67
21 [ file_io       ] read 1024 bytes at 0x8600
22 AAAAAAAAAA...AAAAAAAAB[ fat12          ] --- fat12_close() ---
```

Let's follow the trace from the top. Mounting now reports a data cluster count of 4067, then loads the FAT: 12 sectors starting at LBA 1, exactly one copy, read once into memory. Next, `fat12_open` reads the root directory to find

`BIGFILE.TXT`'s entry. Then come the cluster reads: first cluster 6, then cluster 7, proof that our code followed the chain 6 -> 7 all the way to its end-of-chain marker. Finally, `cat` prints the file's contents: a long run of A's (1024 of them, if you feel like counting) and, at long last, the single B that went missing at the start of this chapter. (With no trailing newline in the file, `fat12_close`'s own trace lands right behind it.)

What's Next

`first_cluster` is only the first stop on a chain we now know how to walk, cluster by cluster, all the way to the last byte of the file. But everything we have built so far still only reaches the root directory. Chapter 6 teaches our code to walk into subdirectories, so listing and reading work at any depth.

Chapter 6: Full Path Walking

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Subdirectories: Files in Disguise

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

The Path Walking Algorithm

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Retrofitting Reads

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

What's Next

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Part II: Writing

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Chapter 7: Writing Files

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Finding a Free Slot

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Encoding the Name

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Encoding the Timestamp

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Linking the File

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Opening a File for Writing

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Buffering the Write

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

include/fat12.h

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Cluster Allocation

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Writing the Data

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Closing the File

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

The create Command

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/cli/main.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/cli/main.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

What's Next

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Chapter 8: Creating Directories

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Assembling the Directory

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

include/fat12.h

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

The **mkdir** Command

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/cli/main.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/cli/main.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

What's Next

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Chapter 9: Deleting

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

When Is a Directory Empty?

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Freeing a Cluster Chain

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Putting It All Together

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

include/fat12.h

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

The rm Command

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/cli/main.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/cli/main.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

What's Next

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Chapter 10: Moving (and Renaming)

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Rename vs. Move

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Moving the Entry

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

`include/fat12.h`

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

`src/fat12.c`

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

The mv Command

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/cli/main.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/cli/main.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

What's Next

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Part III: Advanced

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Chapter 11: Formatting

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

The Blueprint

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

The FAT Size Problem

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

The Boot Sector

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

The FAT Region

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

The Root Directory

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

The Data Region

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Putting It All Together

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

include/fat12.h

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

The format Command

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/cli/main.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/cli/main.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

What's Next

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Chapter 12: Into the Kernel

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

The RAM Disk Block Device

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/kernel/ram_disk_block_device.h

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/kernel/ram_disk_block_device.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Printing to Screen

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/kernel/vga.h

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/kernel/vga.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

The Kernel

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/kernel/main.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Building the Kernel

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/kernel/boot.s

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

kernel.ld

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

build_kernel.sh

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

The Freestanding Environment

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/kernel/lib.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Booting the Kernel

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

What's Next

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Conclusion

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

What's Next

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Thank You

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Appendixes

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Appendix A: Talking to a Real Disk

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

The ATA Controller

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/kernel/ata_block_device.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Port I/O

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/kernel/ata_block_device.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Issuing a Command

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/kernel/ata_block_device.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Opening

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/kernel/ata_block_device.h

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/kernel/ata_block_device.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Reading

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/kernel/ata_block_device.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Writing

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/kernel/ata_block_device.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Completing the Block Device

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/kernel/ata_block_device.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Replacing the RAM Disk

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

platform/kernel/main.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Building and Running

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Appendix B: Long Filename Support (LFN / VFAT)

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Finding the Fragments

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Anatomy of an LFN Entry

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

`src/layout.h`

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

A Quick Look at UTF-16

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Decoding the Fragments

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Reassembling the Name

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

The Checksum

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

src/fat12.c

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

What Full LFN Support Would Take

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Showing the Name

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Resolving a Path

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Creating

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Deleting

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Moving

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Appendix C: The Boot Sector's Leftovers

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Hidden Sectors (Partitioning)

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

CHS (Cylinder-Head-Sector) Addressing

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Media Descriptor

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Extended BPB — Additional Fields

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

OEM Name (Original Equipment Manufacturer)

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Booting from FAT12

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Field Reference

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Appendix D: FAT Family Reference

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

FAT Entry Widths and Values

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Root directory

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

BPB

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Volume Size Limits

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.

Further Reading

This content is not available in the sample book. The book can be purchased on Leanpub at <https://leanpub.com/fat12>.